A Survey

Speaker: Xiang-Jun Ou, Rong-Hao Huang, Jing Wang

Date: 2026/6/23

Methods	Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Laplace-LoRA	Bayesian	Parameter	White-box	Fine-tuning	High	Calibration
BayesPE	Ensemble	Input	White-box	Post-hoc	Low	Calibration/Ranking
Deep Ensembles	Ensemble	Parameter	White-box	Full training	High	Calibration
LoRA Ensembles	Ensemble	Parameter	White-box	Fine tuning	High	Calibration
SPUQ	Consensus-based	Input/Decoding	Black-box	Post-hoc	Low	Calibration
ConU	Consensus-based	Decoding	Black-box	Post-hoc	Low	Coverage/Ranking
SelfCheckGPT	Consensus-based	Decoding	Black-box	Post-hoc	Medium	Ranking
NumSet	Consensus-based	Decoding	Black-box	Post-hoc	Medium	Ranking
EigV	Consensus-based	Decoding	Black-box	Post-hoc	Medium	Ranking
Deg	Consensus-based	Decoding	Black-box	Post-hoc	Medium	Ranking
Ecc	Consensus-based	Decoding	Black-box	Post-hoc	Medium	Ranking
BSDetector	Consensus-based	Decoding	Black-box	Post-hoc	Medium	Ranking
D-UE	Consensus-based	Decoding	Black-box	Post-hoc	Medium	Ranking
CSS	Consensus-based	Decoding	Black-box	Post-hoc	Medium	Ranking
LofreeCP	Consensus-based	Decoding	Black-box	Post-hoc	Medium	Coverage/Ranking
EigenScore	Consensus-based	Decoding	White-box	Post-hoc	Medium	Ranking
Semantic Entropy	Consensus-based	Decoding/Token-level	White-box	Post-hoc	Medium	Ranking
Semantic Density	Consensus-based	Decoding/Token-level	White-box	Post-hoc	Medium	Ranking
SAR	Consensus-based	Decoding/Token-level	White-box	Post-hoc	Medium	Ranking
Topo-UQ	Consensus-based	Reasoning	Black-box	Post-hoc	Medium	Ranking
CoT Entropy	Consensus-based	Reasoning	White-box	Post-hoc	Low	Ranking
CP-MCQA	Single-pass	Token-level	White-box	Post-hoc	Low	Coverage/Ranking
MARS	Single-pass	Token-level	White-box	Post-hoc	Medium	Ranking
CCP	Single-pass	Token-level	White-box	Post-hoc	Medium	Ranking
SEPs	Single-pass	Decoding	White-box	Fine-tuning	High	Ranking
BLL	Single-pass	Decoding	White-box	Fine-tuning	High	Ranking/Calibration
LACIE	Single-pass	Decoding	White-box	Fine-tuning	High	Calibration/Ranking
SaySelf	Single-pass	Reasoning	White-box	Fine-tuning	High	Calibration
Linguistic Calibration	Single-pass	Reasoning	White-box	Fine-tuning	High	Calibration
CoT-UQ	Single-pass	Reasoning/Token-level	White-box	Post-hoc	Medium	Ranking
Verbalized Probability	Single-pass	-	White-box	Fine-tuning	High	Calibration
P(IK)	Single-pass	-	White-box	Fine-tuning	High	Calibration/Ranking
P(True)	Single-pass	-	White-box	Post-hoc	Low	Calibration/Ranking

Bayesian Methods

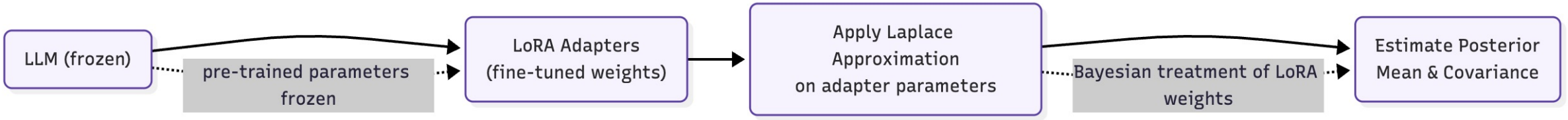
Methods	Papers
Laplace-LoRA	Bayesian Low-rank Adaptation for Large Language Models, arxiv 2023

Laplace-LoRA

Laplace-LoRA[115]

☐ Core Idea

- Fine-tuned LLMs often become overconfident, especially when trained on small datasets. Laplace-LoRA introduces a Bayesian perspective to LoRA fine-tuning by approximating the epistemic uncertainty over LoRA parameters using a Laplace approximation.



[115] Bayesian Low-rank Adaptation for Large Language Models, arxiv 2023

Laplace-LoRA[115]

- **Task Type:** Multiple-choice QA & Binary reasoning

- **Introduction:** Laplace-LoRA applies a Laplace approximation to the low-rank adapter parameters after fine-tuning, introducing a Bayesian posterior over LoRA updates while keeping the backbone fixed. It quantifies parameter uncertainty and provides a distributional view of LoRA-based adaptation within standard LLM fine-tuning workflows.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Bayesian	Parameter	White-box	Post-hoc	High	Ranking/Calibration

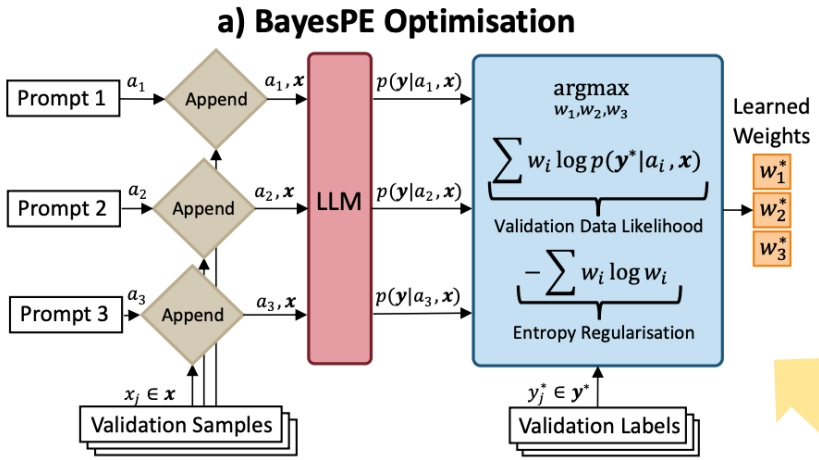
[115] Bayesian Low-rank Adaptation for Large Language Models, arxiv 2023

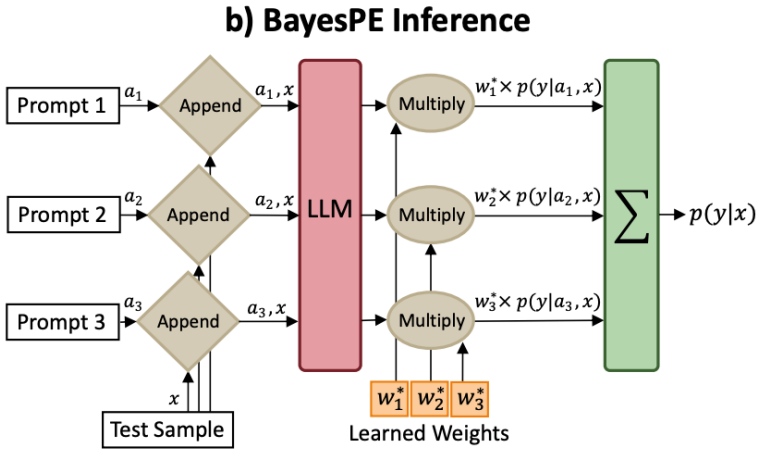
Ensemble Methods

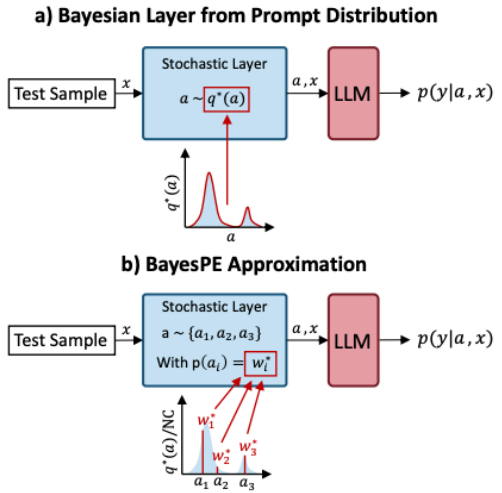
Methods	Papers
BayesPE	Bayesian Prompt Ensembles: Model Uncertainty Estimation for Black-Box Large Language Models, ACL 2024
Deep Ensembles	Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles, Neurips 2016
LoRA Ensembles	LoRA ensembles for large language model fine-tuning, arxiv 2023

BayesPE

BayesPE[95]







$$\underset{\mathbf{w}}{\operatorname{argmax}} \underbrace{\sum_{i}^{N} w_i \log p(\mathbf{y}^*|a_i, \mathbf{x})}_{\text{Validation Data Likelihood}} \underbrace{- \sum_{i}^{N} w_i \log w_i}_{\text{Entropy}}$$

Loss

Regularization Term

☐ Pipeline:

(a) BayesPE optimisation: Each validation sample $x_j \in \mathbf{x}$ is appended to each semantically equivalent prompt a_i, and the LLM computes the likelihood $p(y_j^*|a_i, x_j)$ for the ground-truth label $y_j^* \in \mathbf{y}^*$; these likelihoods are used to learn prompt weights w_i^* via the BayesPE objective.

(b) BayesPE inference: For a test sample x, the LLM produces prompt-conditioned distributions $p(y|a_i, x)$, which are weighted by the learned weights w_i^* and aggregated to obtain the final distribution $p(y \mid x)$.

[95] Bayesian Prompt Ensembles: Model Uncertainty Estimation for Black-Box Large Language Models, ACL 2024

BayesPE[95]

- **Task Type:** Open-ended generation

- **Introduction:** Bayesian Prompt Ensembles (BayesPE) approximate Bayesian inference by sampling multiple prompt perturbations to induce posterior diversity in LLM outputs. It learns prompt-specific weights to aggregate multi-prompt predictions at inference time, yielding calibrated uncertainty estimates. BayesPE provides a lightweight, parameter-free Bayesian approximation for large language models.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Ensemble	Input	White-box	Post-hoc	Low	Calibration

[95] Bayesian Prompt Ensembles: Model Uncertainty Estimation for Black-Box Large Language Models, ACL 2024

Deep Ensembles

Deep Ensembles[52]

Deep Ensembles approximate uncertainty by training multiple independent networks and averaging their predictions.

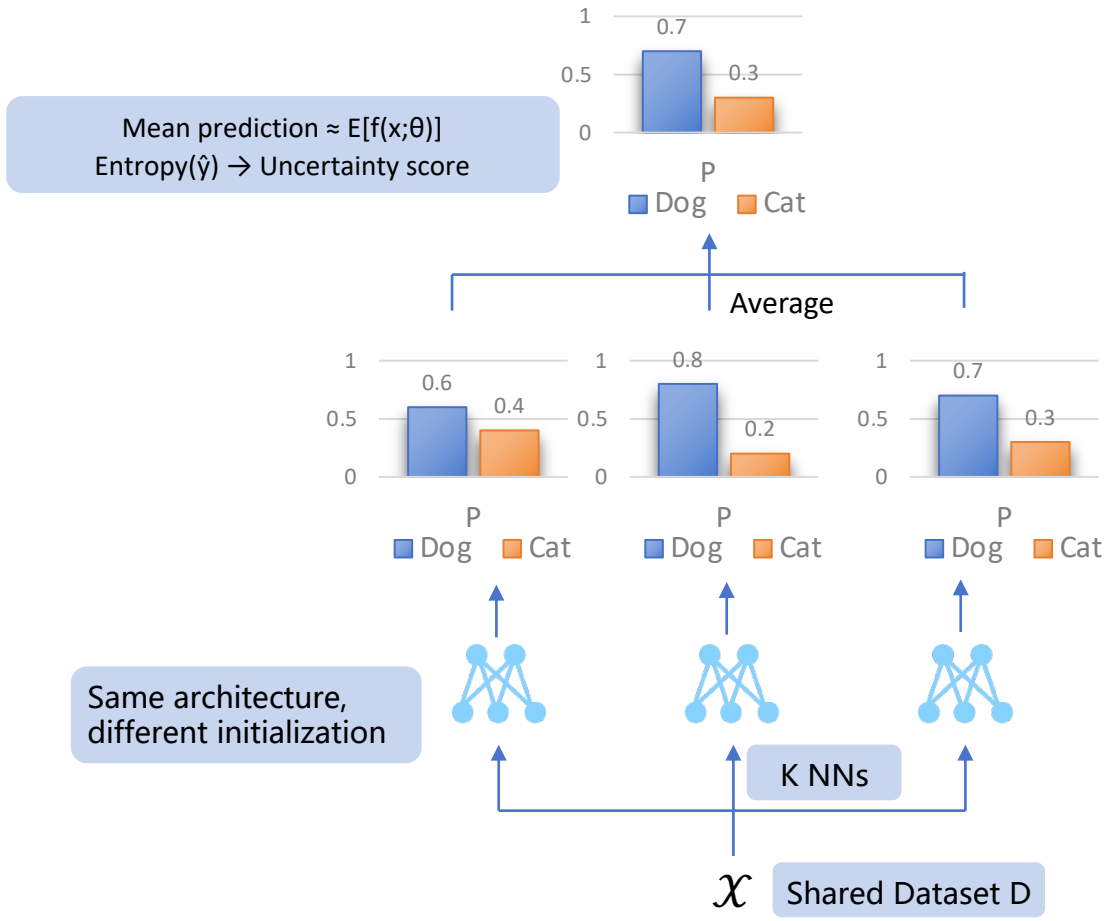
- **Training and Inference Pipeline**
  - Initialize K independent models with different random seeds.
  - Train them in parallel on the same dataset: $\{\theta_1, \theta_2, .., .. \theta_K\}$.
  - For an input x:

$$y_i = f(x; \theta_i), i \in [K]$$

  - Compute ensemble mean and entropy:

$$\hat{y} = \frac{1}{K}\sum_i y_i \,, \hat{H} = \text{Entropy}(\hat{y})$$

-> Higher entropy = higher uncertainty.



[52] Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles, Neurips 2016

Deep Ensembles[52]

- **Task Type:** Regression / Classification task

- **Introduction:** Deep Ensembles approximate uncertainty by training multiple independent networks with different initializations and averaging their predictions. It quantifies parameter uncertainty and establishes a reliable non-Bayesian baseline in modern UQ.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Ensemble	Parameter	White-box	Full training	High	Calibration/Ranking

[52] Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles, Neurips 2016

LoRA Ensembles

LoRA Ensembles[106]

LoRA Ensembles replaces the costly multi-model training in traditional Deep Ensembles with multiple Low-Rank Adapters (LoRA) attached to a shared base pre-trained model

FineTune Procedure

- Each adapter learns a low-rank update to the frozen base model weights:

$$\Delta W = \alpha BA \qquad B \in \mathbb{R}^{d\times r} \quad A \in \mathbb{R}^{r\times k}$$

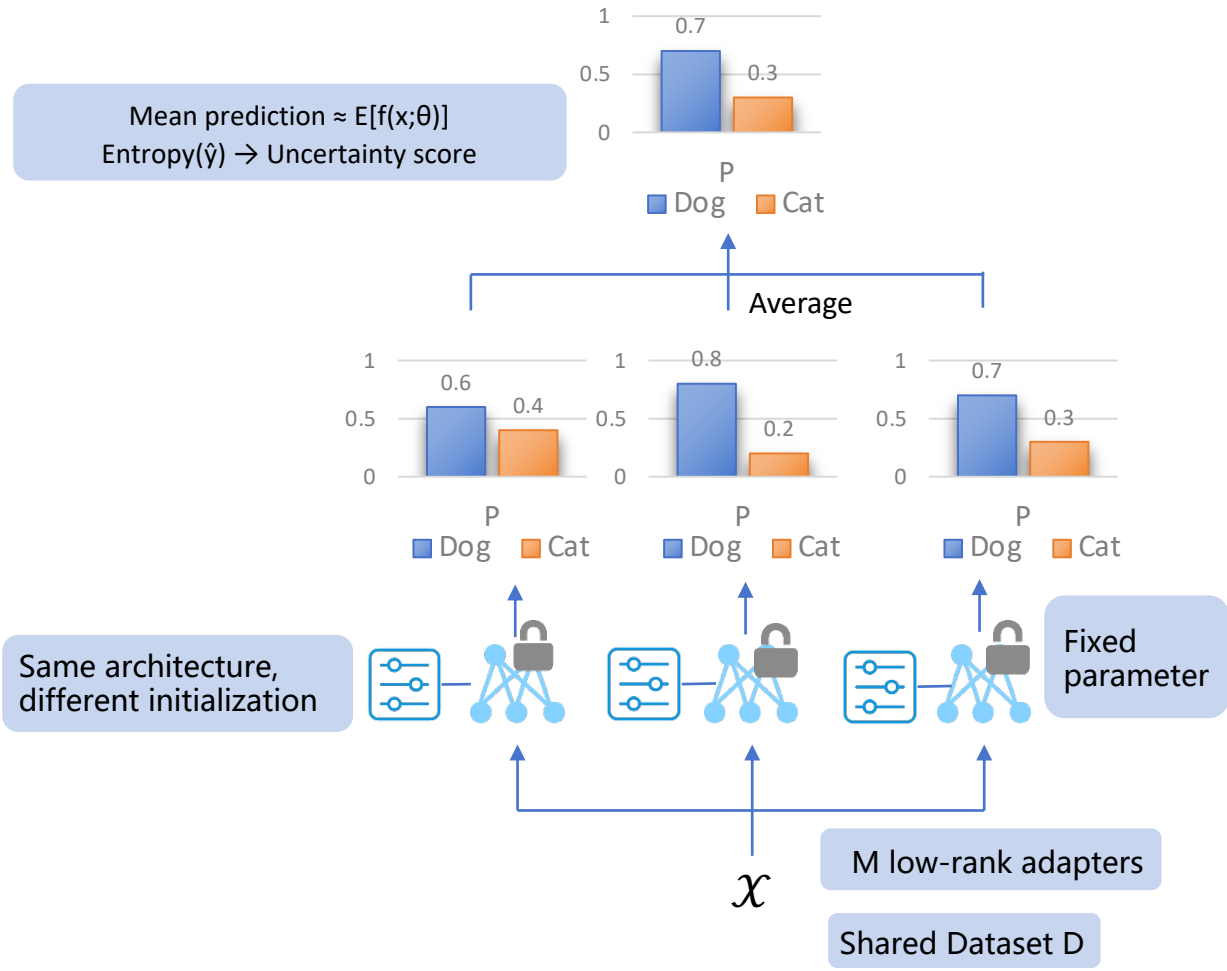
$$L(\Delta W) = \min_{\Delta W} \sum_{n} -\log p(y_n|x_n; W^* + \Delta W)$$

- Initialize A randomly, $\mathrm{B} = 0$;
- KL regularization, strong weight decay, early stopping

Ensemble Aggregation

- Final prediction combines M adapters:

$$p_{ens}(y|x) = \frac{1}{M}\sum_{m=1}^{M} p(y|x; W_m)$$



[106] LoRA ensembles for large language model fine-tuning, arxiv 2023

LoRA Ensembles[106]

- **Task Type:** Multiple-choice QA & Binary classification

- **Introduction:** LoRA Ensembles adapts Deep Ensembles by attaching multiple independently initialized low-rank adapters to a shared LLM backbone and averaging their token-level distributions. It quantifies parameter uncertainty and provides a parameter-efficient ensembling strategy within standard LLM fine-tuning workflows.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Ensemble	Parameter	White-box	Fine-tuning	High	Calibration

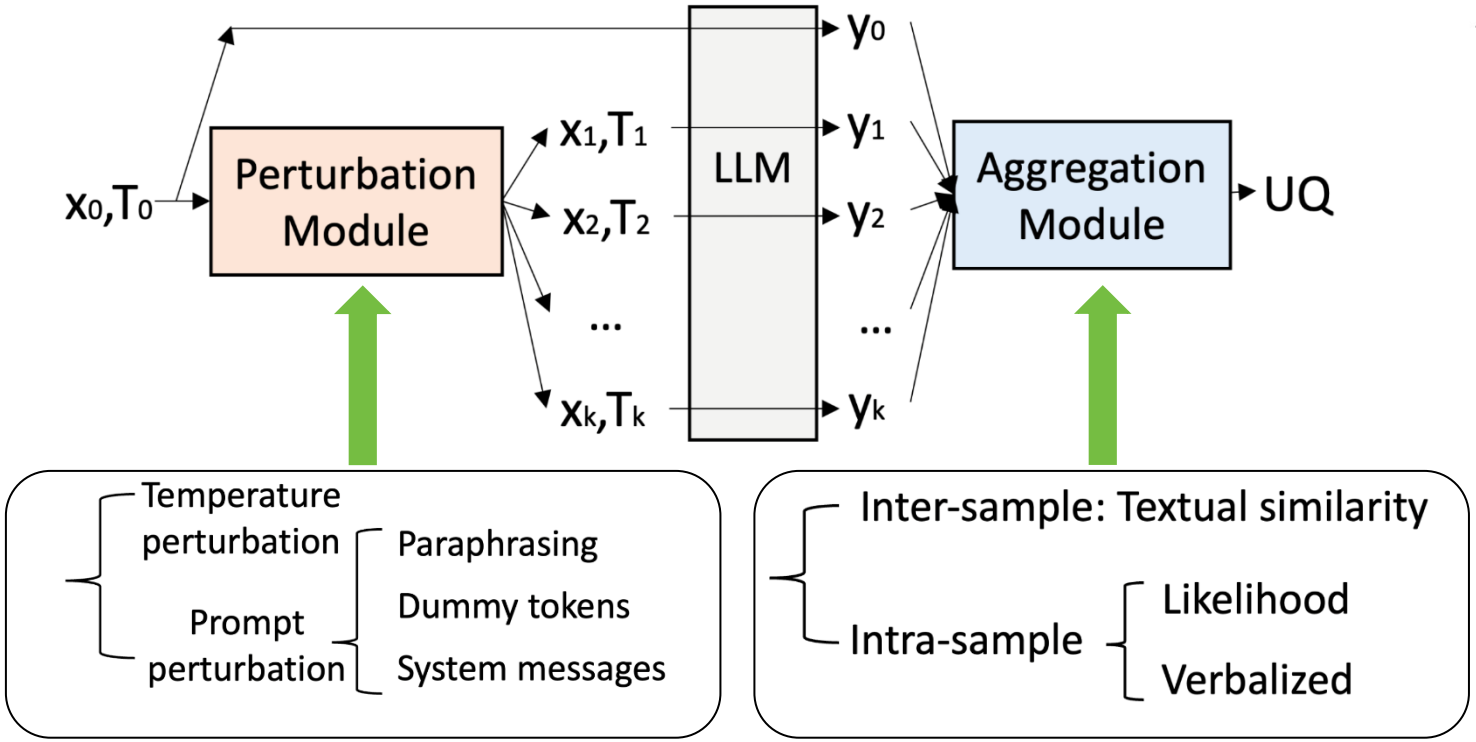
[106] LoRA ensembles for large language model fine-tuning, arxiv 2023

Consensus-based Methods

Methods	Papers
SPUQ	SPUQ: Perturbation-Based Uncertainty Quantification for Large Language Models, EACL 2024
ConU	ConU: Conformal Uncertainty in Large Language Models with Correctness Coverage Guarantees, EMNLP 2024
SelfCheckGPT	SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models, EMNLP 2023
NumSet	Generating with Confidence: Uncertainty Quantification for Black-box Large Language Models, TMLR 2024
EigV	Generating with Confidence: Uncertainty Quantification for Black-box Large Language Models, TMLR 2024
Deg	Generating with Confidence: Uncertainty Quantification for Black-box Large Language Models, TMLR 2024
Ecc	Generating with Confidence: Uncertainty Quantification for Black-box Large Language Models, TMLR 2024
BSDetector	Quantifying Uncertainty in Answers from any Language Model and Enhancing their Trustworthiness, ACL 2024
D-UE	LLM uncertainty quantification through directional entailment graph and claim level response augmentation, arxiv 2024
CSS	CSS: Contrastive Semantic Similarity for Uncertainty Quantification of LLMs, arxiv 2024
LofreeCP	API Is Enough: Conformal Prediction for Large Language Models Without Logit-Access, EMNLP 2024
EigenScore	INSIDE: LLMs' Internal States Retain the Power of Hallucination Detection, ICLR 2024
Semantic Entropy	Detecting hallucinations in large language models using semantic entropy, Nature 2024
Semantic Density	Semantic Density: Uncertainty Quantification for Large Language Models through Confidence Measurement in Semantic Space, Neurips 2024
SAR	Shifting Attention to Relevance: Towards the Predictive Uncertainty Quantification of Free-Form Large Language Models, ACL 2024
Topo-UQ	Understanding the Uncertainty of LLM Explanations: A Perspective Based on Reasoning Topology, COLM 2025
CoT Entropy	Uncertainty-aware Step-wise Verification with Generative Reward Models, arxiv 2025

SPUQ

SPUQ[35]



Algorithm 1 SPUQ

Require: Original temperature T_0, original prompt x_0, perturbation and aggregation module hyperparameters (Fig. 2)

Ensure: Confidence score c

1: Obtain k perturbed inputs (T_i, x_i) from the perturbation module, where $i = 1$ to k.
2: **for** $j = 0$ to k **do**
3: Send (T_j, x_j) to LLM being tested
4: Obtain output y_j from LLM
5: **end for**
6: Send set $\{y_j\}$ to the aggregation module
7: Obtain the confidence score c
8: **return** c

- **Core Idea:**
  - accurately reflect the uncertainty of LLMs by perturbing the inputs and sampling the outputs.

Input x_i	**Prediction** y_i	**Likelihood**
Original		
Will Jay-Z reach the age of 60 before Kendrick Lamar?	No *(Incorrect)*	92%
Perturbed prompt		
Is it likely that Jay-Z will turn 60 before Kendrick Lamar does?	Yes	83%
Will Jay-Z hit the age of 60 before Kendrick Lamar does?	No	97%
Before Kendrick Lamar, will Jay-Z reach the age of 60?	Yes	94%
Is it possible that Jay-Z will turn 60 before Kendrick Lamar?	No	95%

[35] SPUQ: Perturbation-Based Uncertainty Quantification for Large Language Models, EACL 2024

SPUQ[35]

- **Task Type:** Open-ended generation

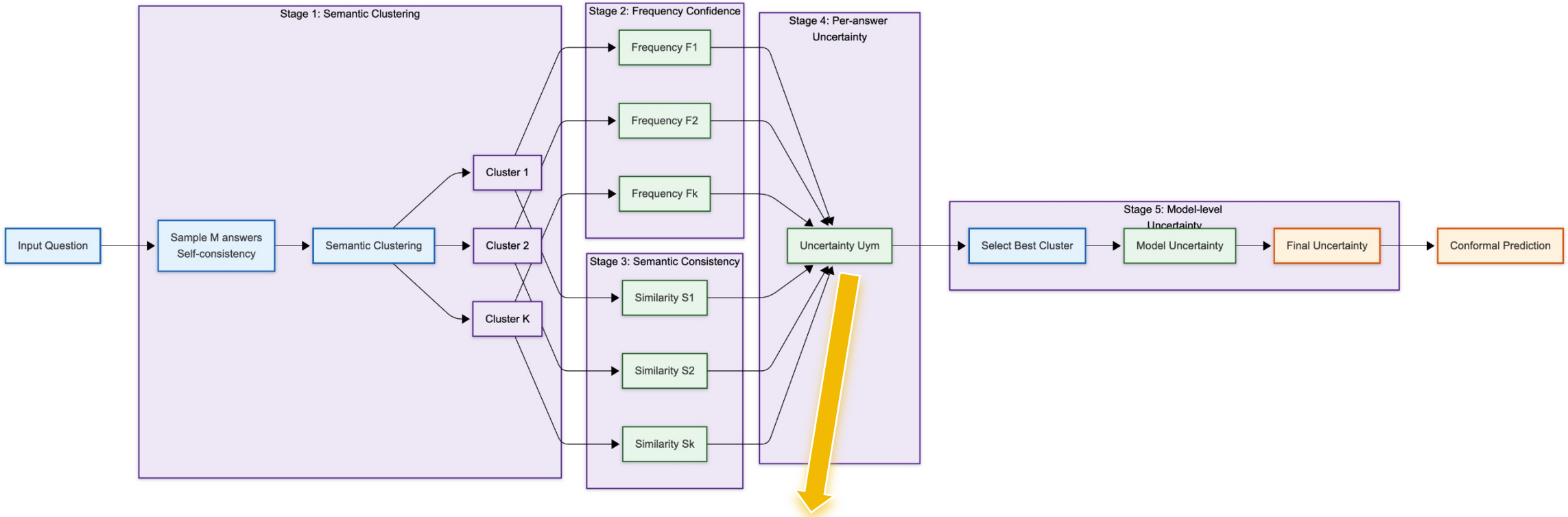
- **Introduction:** Sampling with perturbation for UQ (SPUQ) estimates uncertainty by perturbing the input and sampling multiple outputs. It entails generating a set of perturbations for LLM inputs, sampling outputs for each perturbation, and incorporating an aggregation module to unify these signals into a generalizable uncertainty estimate.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Consensus-based	Input/Decoding	Black-box	Post-hoc	Low	Calibration

[35] SPUQ: Perturbation-Based Uncertainty Quantification for Large Language Models, EACL 2024

ConU

ConU[108]



ConU measures uncertainty by two complementary components: $\mathcal{F}\left(\hat{y}_k^{(i)}\right) = \frac{V_k}{M}$.

- **Frequency Confidence (F):** How often a semantic meaning appears.
- **Semantic Consistency (S):** How similar each meaning is to others.

$$U\left(\hat{y}_m^{(i)}\right) = 1 - \lambda F\left(\hat{y}_m^{(i)}\right) - (1-\lambda)\frac{1}{K}\sum_{k=1}^{K} S\left(\hat{y}_m^{(i)}, \hat{y}_k^{(i)}\right) F\left(\hat{y}_k^{(i)}\right)$$

cross-encoder model

with $\lambda = 0.5$.

[108] ConU: Conformal Uncertainty in Large Language Models with Correctness Coverage Guarantees, EMNLP 2024

ConU[108]

- **Task Type:** Open-ended generation

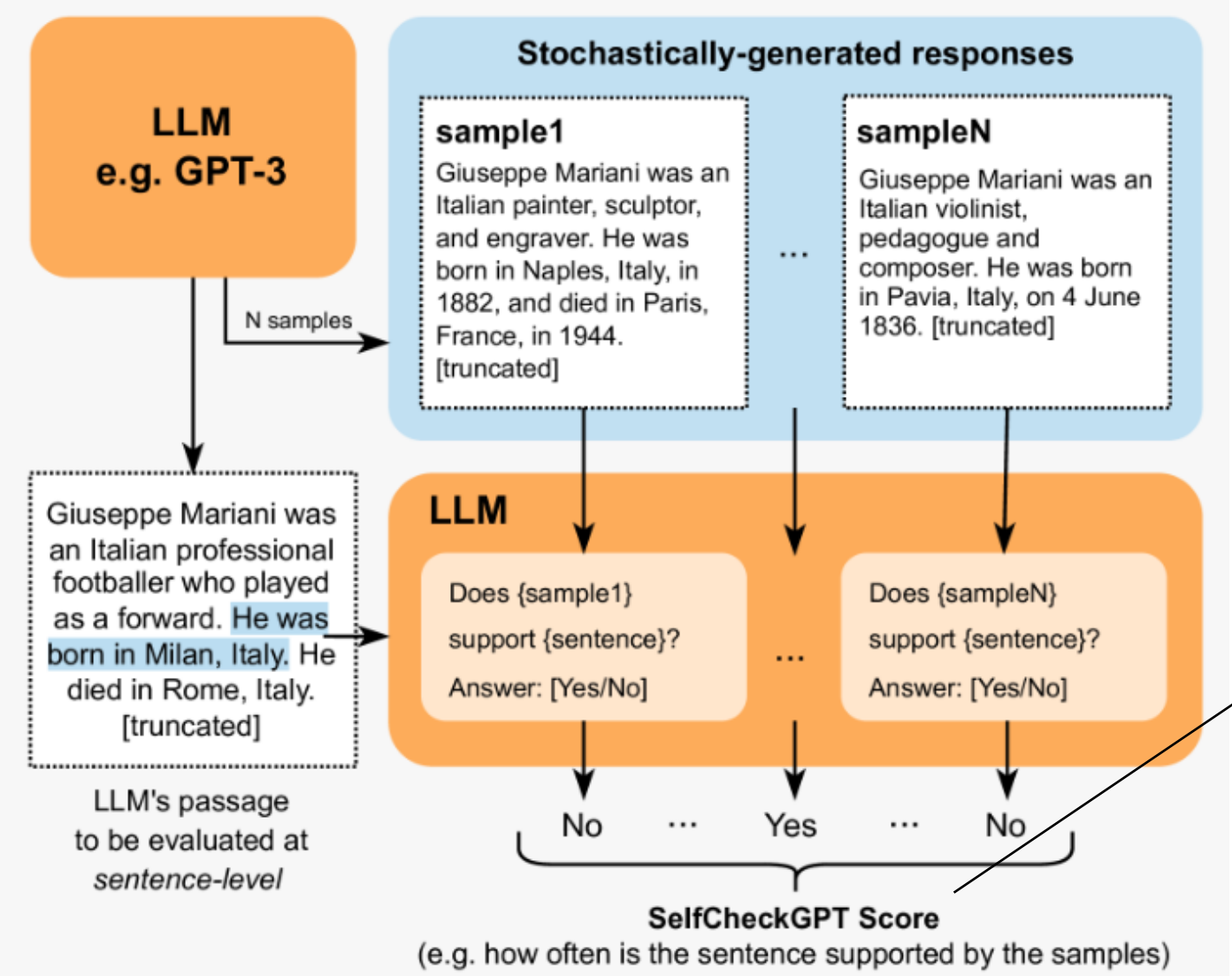
- **Introduction:** ConU estimates uncertainty by sampling multiple generations, clustering their semantics, and computing a frequency and semantics based uncertainty score aligned with correctness. It quantifies prediction uncertainty and offers a black-box way to integrate decoding uncertainty into conformal prediction for correctness coverage.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Consensus-based	Decoding	Black-box	Post-hoc	Low	Ranking

[108] ConU: Conformal Uncertainty in Large Language Models with Correctness Coverage Guarantees, EMNLP 2024

SelfCheckGPT

SelfCheckGPT[70]



x_i^n：{Yes：0.0，No：1.0，N/A：0.5}

$$\mathcal{S}_{\text{Prompt}}(i) = \frac{1}{N}\sum_{n=1}^{N} x_i^n$$

[70] SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models, EMNLP 2023

SelfCheckGPT[70]

- **Task Type:** Open-ended generation
- **Introduction:** LLM generates one target answer and multiple sample answers. It then evaluates the consistency between each sample answer and the target answer. The judgment results are mapped to numerical scores, and their average is taken as the final uncertainty score.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Consensus-based	Decoding	Black-box	Post-hoc	Medium	Ranking

[70] SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models, EMNLP 2023

NumSet

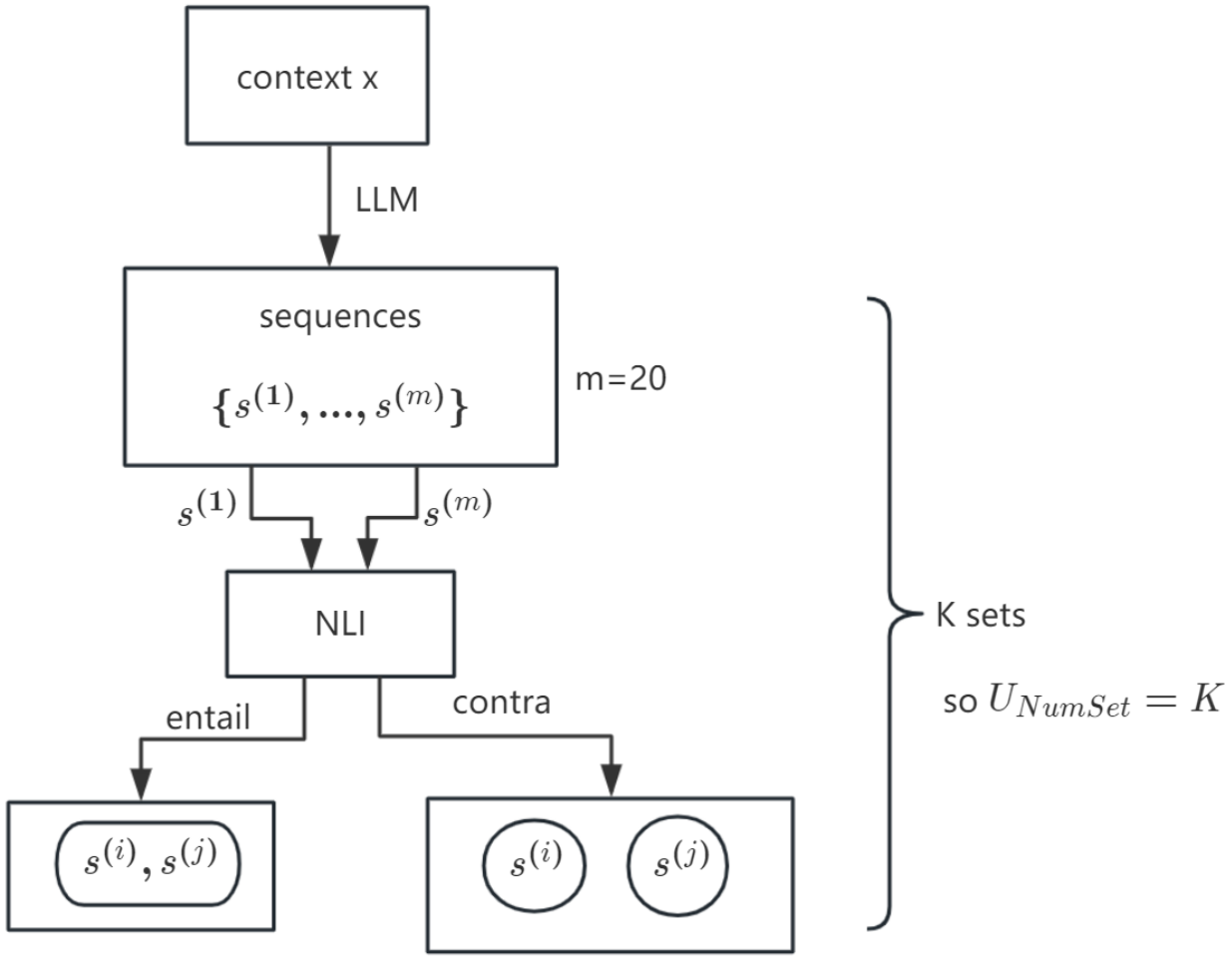
NumSet[60]

Core Idea:

Only use the “number of semantically equivalent classes” as the uncertainty indicator

Measuring Response Similarities:

NLI(DeBERTa-large model)



[60] Generating with Confidence: Uncertainty Quantification for Black-box Large Language Models, TMLR 2024

NumSet[60]

- **Task Type:** Open-ended generation
- **Introduction:** LLM generates many answers. Then a NLI model groups semantically equivalent answers into the same set. The uncertainty score is the number of semantic sets.

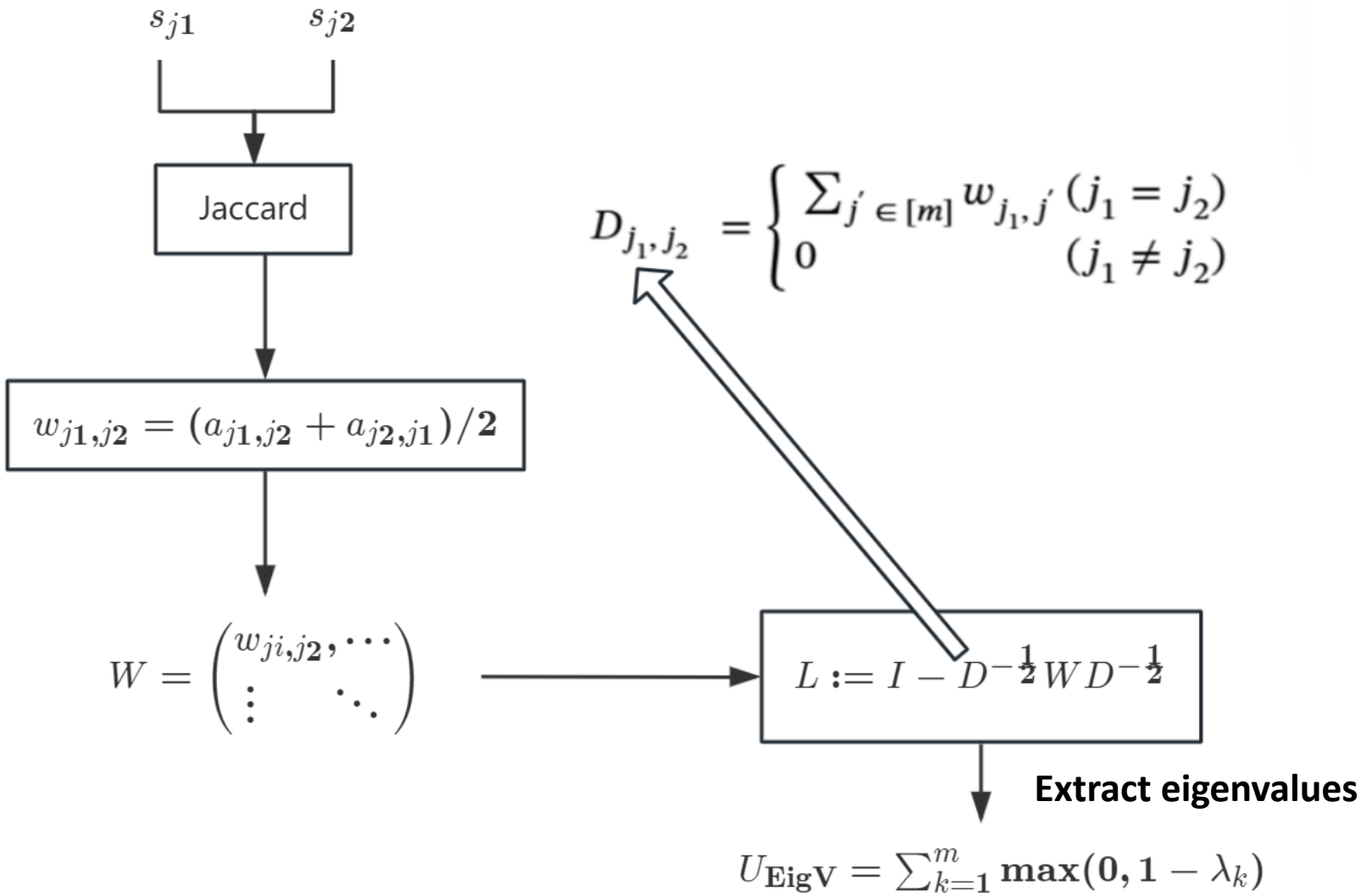
Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Consensus-based	Decoding	Black-box	Post-hoc	Medium	Ranking

[60] Generating with Confidence: Uncertainty Quantification for Black-box Large Language Models, TMLR 2024

EigV

☐Measuring Response Similarities：

Jaccard Similarity/NLI



[60] Generating with Confidence: Uncertainty Quantification for Black-box Large Language Models, TMLR 2024

EigV[60]

- **Task Type:** Open-ended generation

- **Introduction:** After the LLM generates multiple answers, a weighted graph is constructed based on their semantic similarities, and the normalized Laplacian matrix is computed. The eigenvalues of this matrix are then used to calculate the final uncertainty score.

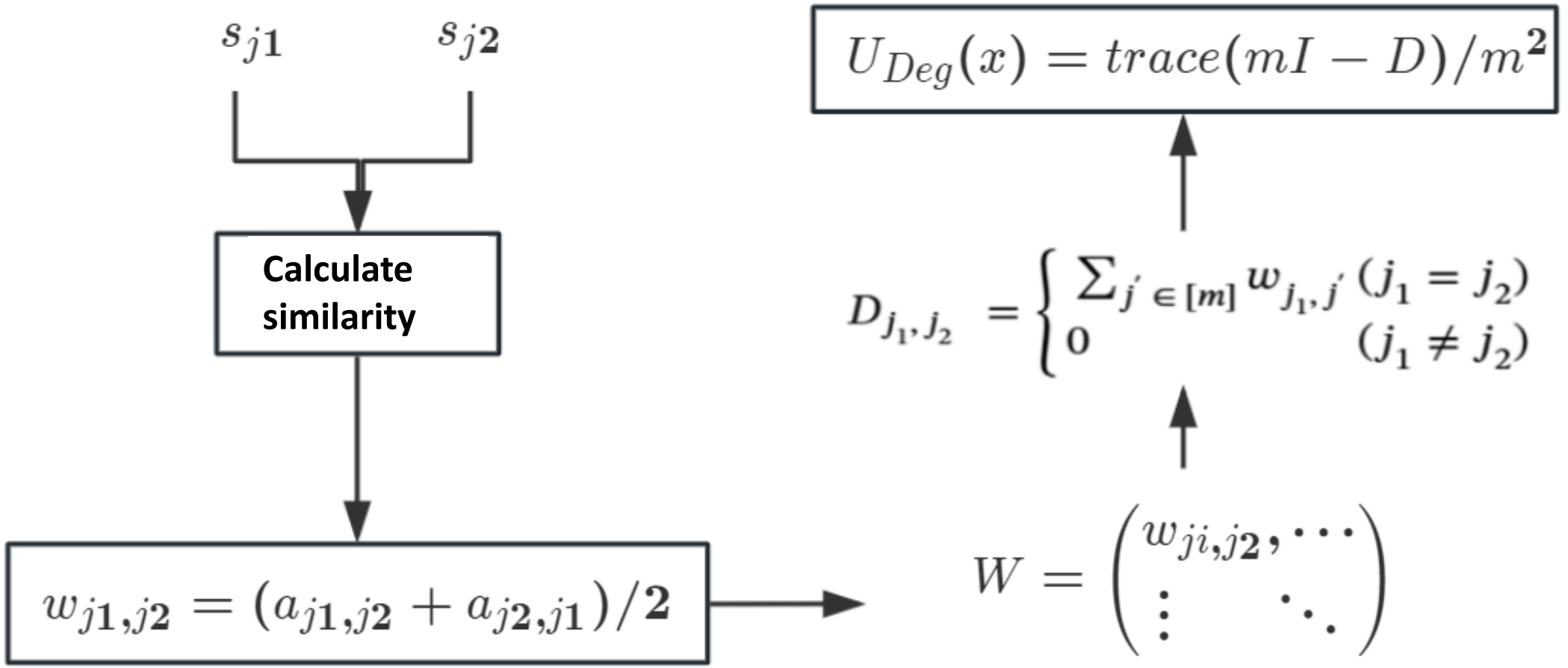
Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Consensus-based	Decoding	Black-box	Post-hoc	Medium	Ranking

[60] Generating with Confidence: Uncertainty Quantification for Black-box Large Language Models, TMLR 2024

Deg

☐Measuring Response Similarities：

Jaccard Similarity/NLI



[60] Generating with Confidence: Uncertainty Quantification for Black-box Large Language Models, TMLR 2024

Deg[60]

- **Task Type:** Open-ended generation
- **Introduction:** The Degree Matrix (Deg) constructs a graph by treating each answer sample as a node and the similarity between two answers as the edge weight, then compute the uncertainty score by calculating the degree of each answer.

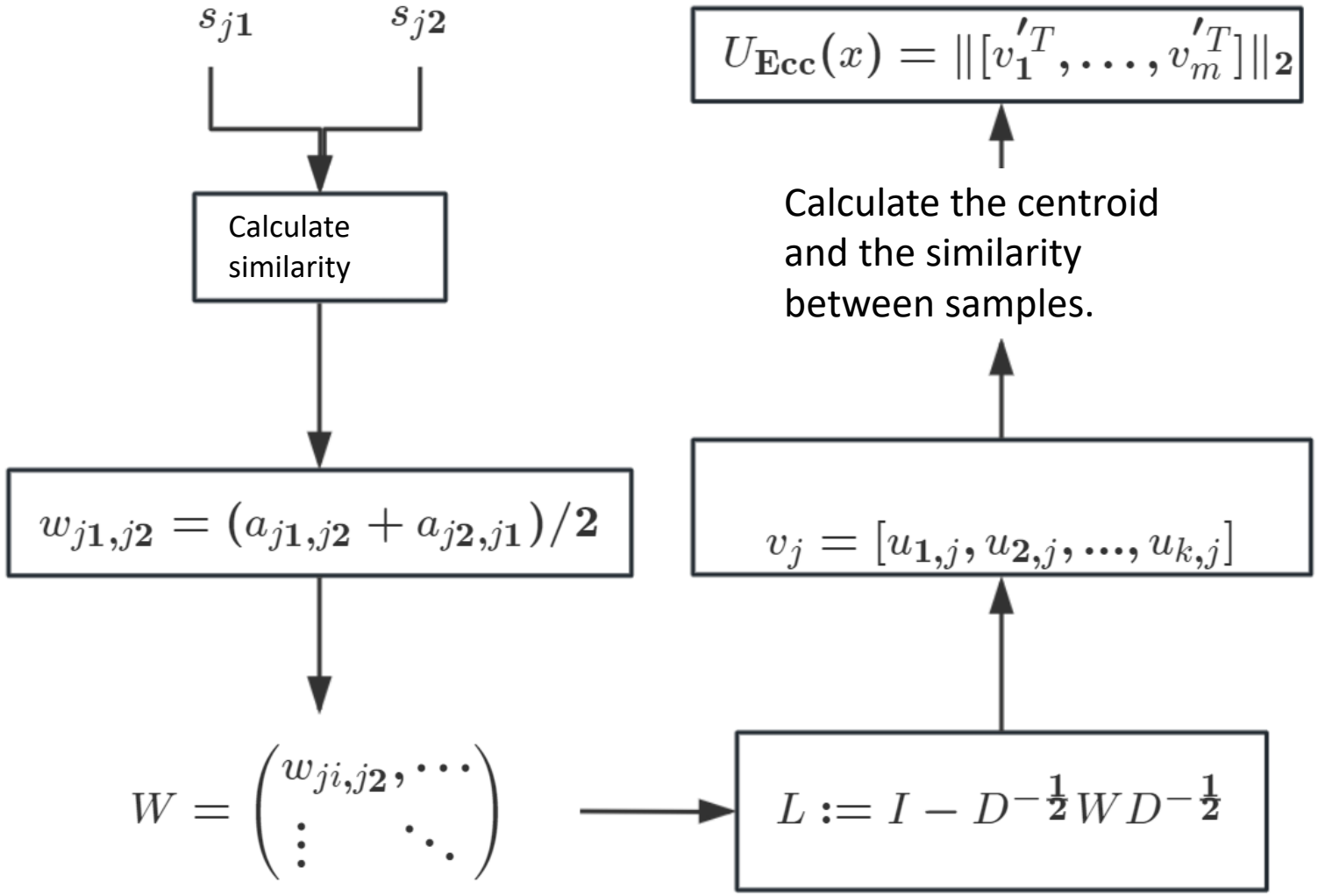
Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Consensus-based	Decoding	Black-box	Post-hoc	Medium	Ranking

[60] Generating with Confidence: Uncertainty Quantification for Black-box Large Language Models, TMLR 2024

Ecc

☐Measuring Response Similarities：

Jaccard Similarity/NLI



[60] Generating with Confidence: Uncertainty Quantification for Black-box Large Language Models, TMLR 2024

Deg[60]

- **Task Type:** Open-ended generation

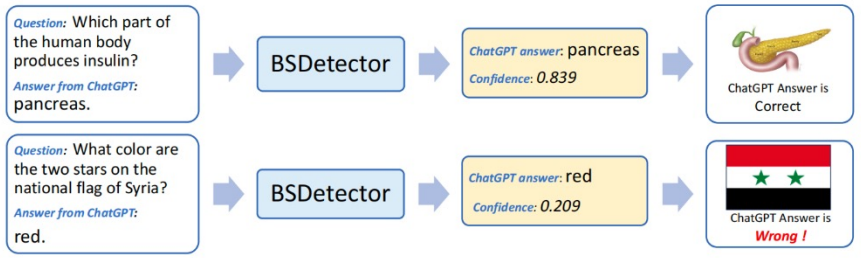
- **Introduction:** Eccentricity (Ecc) is calculated by computed eigenvectors of the graph Laplacian matrix, generating coordinates for each answer in a low-dimensional space. Uncertainty is then quantified by calculating the distance from each point to the center which is the average of all point coordinates.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Consensus-based	Decoding	Black-box	Post-hoc	Medium	Ranking

[60] Generating with Confidence: Uncertainty Quantification for Black-box Large Language Models, TMLR 2024

BSDetector

BSDetector[14]



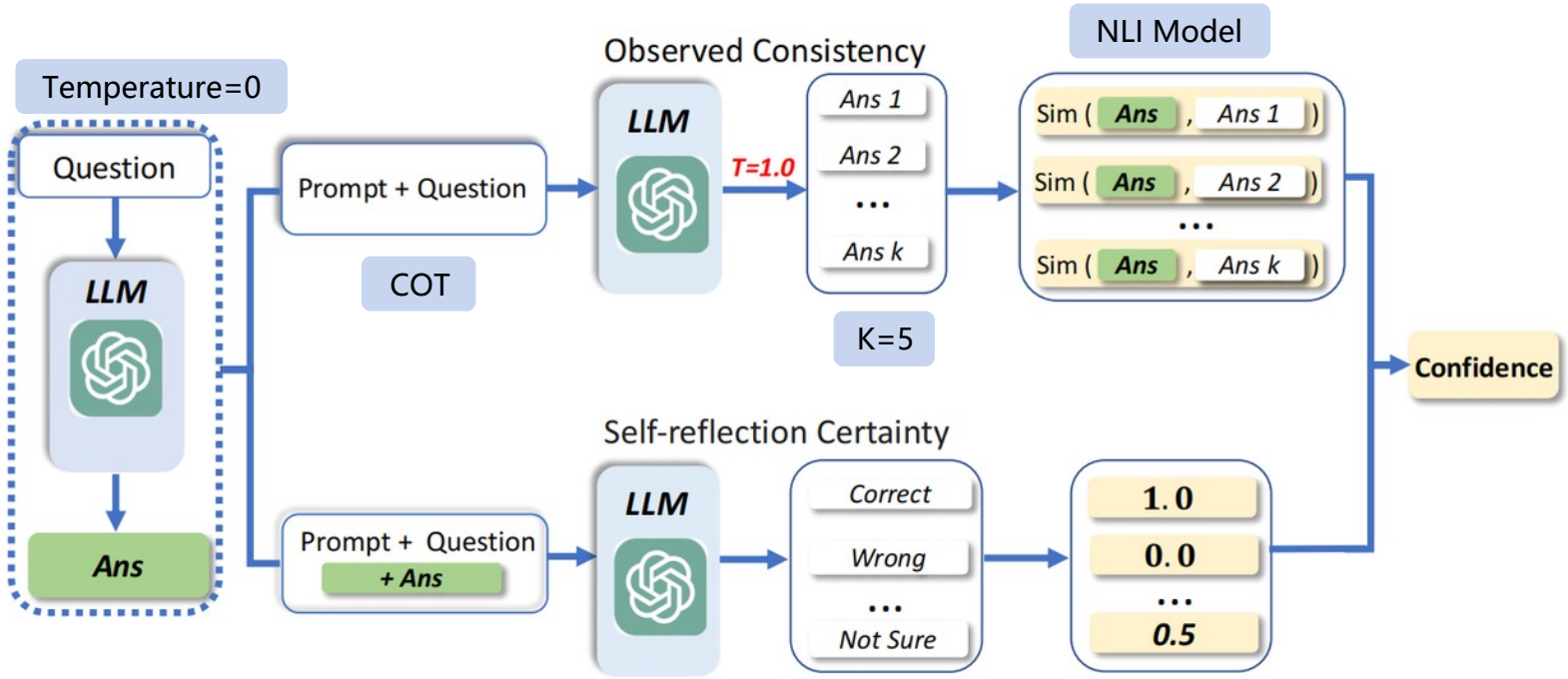
☐ Pipeline Overview

Two complementary estimators are combined into a single confidence score:

$$C = \beta O + (1 - \beta)S$$

where

- O: Observed Consistency (extrinsic)
- S: Self-reflection Certainty (intrinsic)
- $\beta = 0.7$



☐ Observed Consistency

$$s_i = \frac{1}{2}(1 - p_{contradiction} + 1 - p'_{contradiction}), i = 1,2,\dots,k$$

$$o_i = \alpha s_i + (1 - \alpha) r_i, \alpha = 0.8$$

$$O = o_i$$

☐ Self-reflection Certainty

"Is your previous answer correct?

A) Correct B) Incorrect C) I'm not sure."

Scores are assigned as

$$A = 1.0, B = 0.0, C = 0.5$$

and averaged over multiple follow-ups to form S.

[14] Quantifying Uncertainty in Answers from any Language Model and Enhancing their Trustworthiness, ACL 2024

BSDetector[14]

- **Task Type:** Open-ended generation

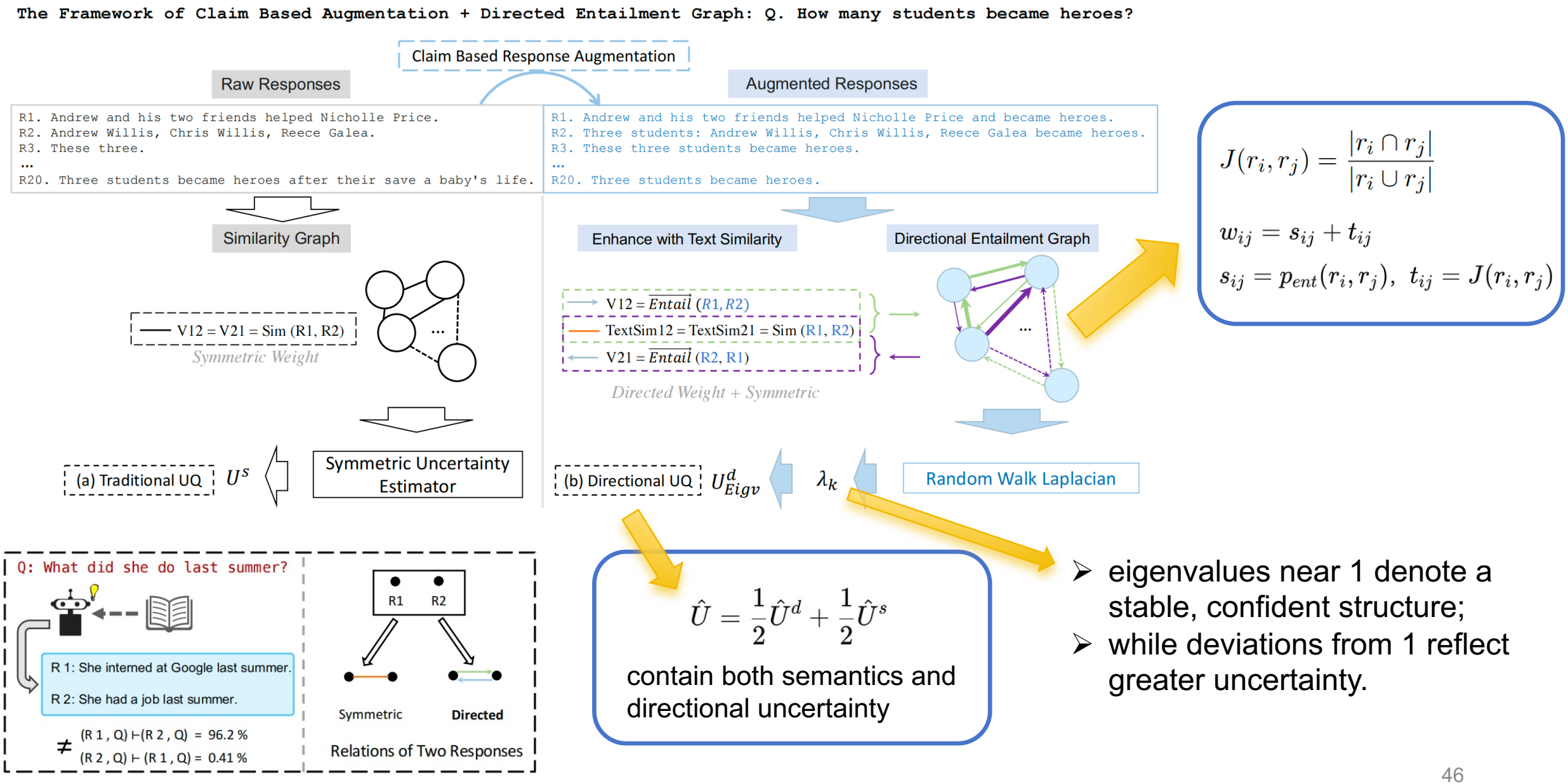
- **Introduction:** BSDetector combines extrinsic semantic similarity across sampled responses with intrinsic self-reflection scores from the LLM to compute a confidence estimate for any generated answer. It quantifies decoding uncertainty and provides a black-box procedure for assessing the reliability of LLM outputs without accessing internal probabilities or model weights.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Consensus-based	Decoding	Black-box	Post-hoc	Medium	Ranking

[14] Quantifying Uncertainty in Answers from any Language Model and Enhancing their Trustworthiness, ACL 2024

D-UE

D-UE[21]



[21] LLM uncertainty quantification through directional entailment graph and claim level response augmentation, arxiv 2024

D-UE[21]

- **Task Type:** Open-ended generation

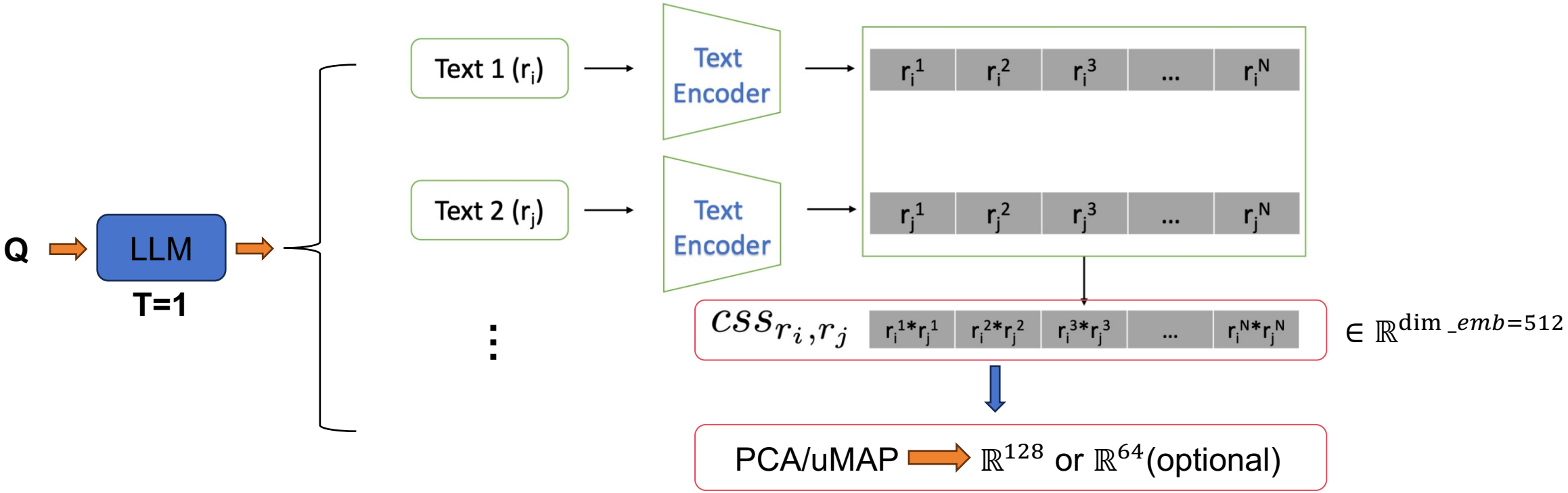
- **Introduction:** D-UE constructs a directed entailment graph over sampled responses, augmenting each answer with claim-level expansions and applying a Random Walk Laplacian to derive an eigenvalue-based uncertainty score. It quantifies decoding uncertainty and provides a structured black-box evaluation method that incorporates directional entailment and semantic inconsistency.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Consensus-based	Decoding	Black-box	Post-hoc	Medium	Ranking

[21] LLM uncertainty quantification through directional entailment graph and claim level response augmentation, arxiv 2024

CSS

CSS[3]



Core Idea:

- to estimate uncertainty for black-box LLMs by leveraging CLIP-based contrastive similarity features.

$$U_{\text{Ecc}}^{css} = \left\| \left[\mathbf{e}^{\mathbf{css}'}{}_1^{\top}, \ldots, \mathbf{e}^{\mathbf{css}'}{}_m^{\top} \right] \right\|_2$$

$$U_{\text{Eig}}^{css} = \sum_{k=1}^{m} \max\left(0, 1 - \lambda_k^{css}\right)$$

$$U_{\text{Deg}}^{css} = \text{trace}(m - D^{css}) / m^2$$

[3] CSS: Contrastive Semantic Similarity for Uncertainty Quantification of LLMs, arxiv 2024

CSS[3]

- **Task Type:** Open-ended generation

- **Introduction:** Contrastive Semantic Similarity (CSS) extends semantic clustering by extracting CLIP-based text-pair features and applying spectral graph Laplacian clustering to estimate uncertainty. It quantifies decoding uncertainty in black-box LLMs, providing a semantic-consistency signal for selective generation and evaluation.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Consensus-based	Decoding	Black-box	Post-hoc	Medium	Ranking

[3] CSS: Contrastive Semantic Similarity for Uncertainty Quantification of LLMs, arxiv 2024

LofreeCP

LofreeCP[91]

- **Methodology：**
  - **Frequency**
  - **NE**
  - **SS**
- **Steps:**
  - **Frequency：**

$$F(\hat{y}_a^{(i)}, m) = \frac{\tilde{p}[\hat{y}_a^{(i)}]}{m},$$

  - **NE：**

$$H(x^{(i)}|\{\hat{y}_j^{(i)}\}_{j=1}^m) = |\frac{\sum_{a=1}^n \tilde{F}(\hat{y}_a^{(i)}) \log(\tilde{F}(\hat{y}_a^{(i)}))}{\log m}|$$

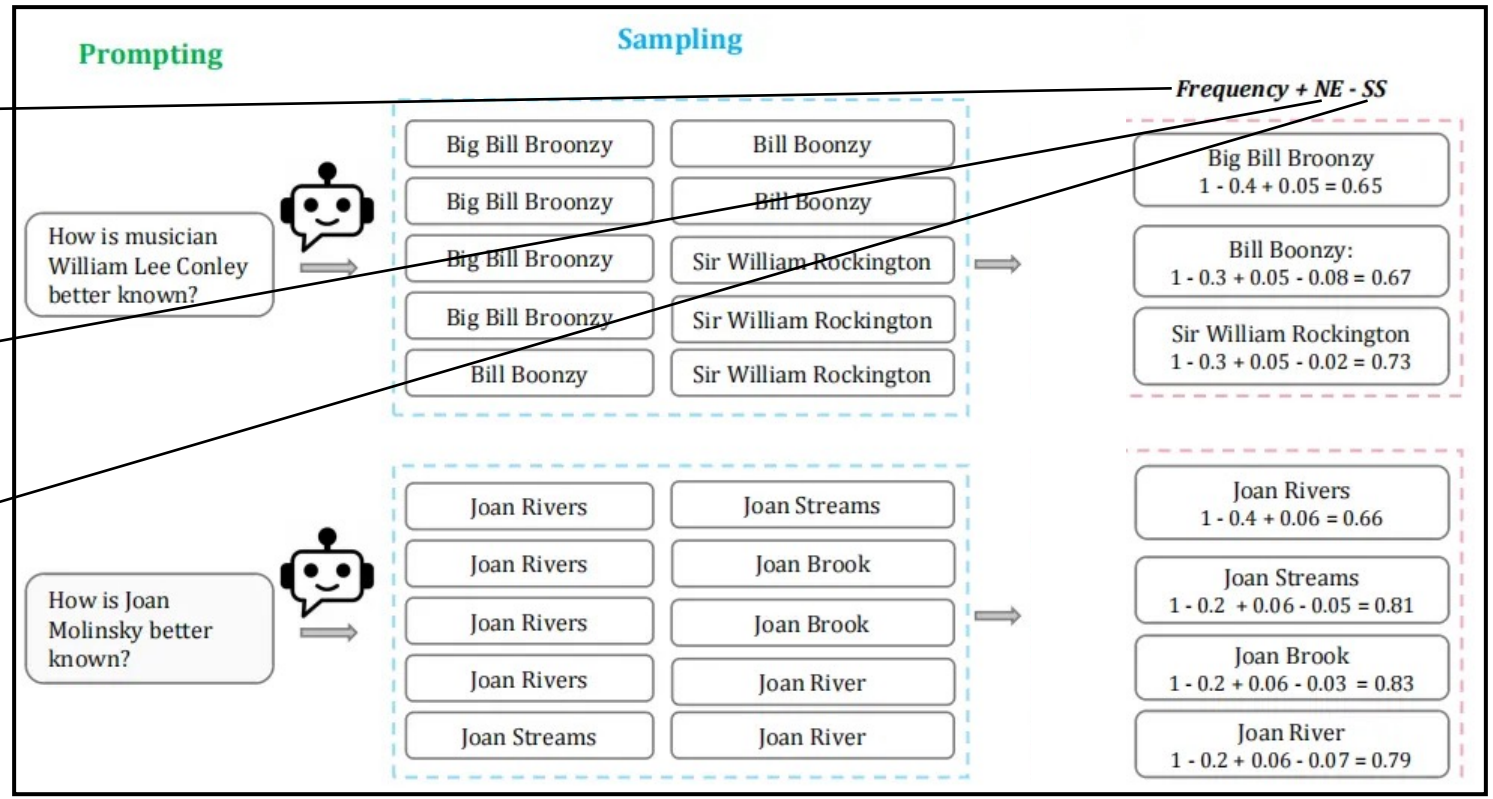
  - **SS：**

$$SS(\hat{y}_a^{(i)}, P_{\text{highest}}^{(i)}) = \frac{\mathbf{v}(\hat{y}_a^{(i)}) \cdot \mathbf{v}(P_{\text{highest}}^{(i)})}{\|\mathbf{v}(\hat{y}_a^{(i)})\| \cdot \|\mathbf{v}(P_{\text{highest}}^{(i)})\|}$$

The same as S（X，Y）

$$N^{(i)} = -F(\hat{y}_a^{(i)}, m) + \lambda_1 \cdot H(x^{(i)}|\{\hat{y}_j^{(i)}\}_{j=1}^m) - \lambda_2 \cdot SS(\hat{y}_a^{(i)}, P_{\text{highest}}^{(i)}),$$

Frequency　NE　SS



[91] API Is Enough: Conformal Prediction for Large Language Models Without Logit-Access, EMNLP 2024

LofreeCP[91]

- **Task Type:** Open-ended generation

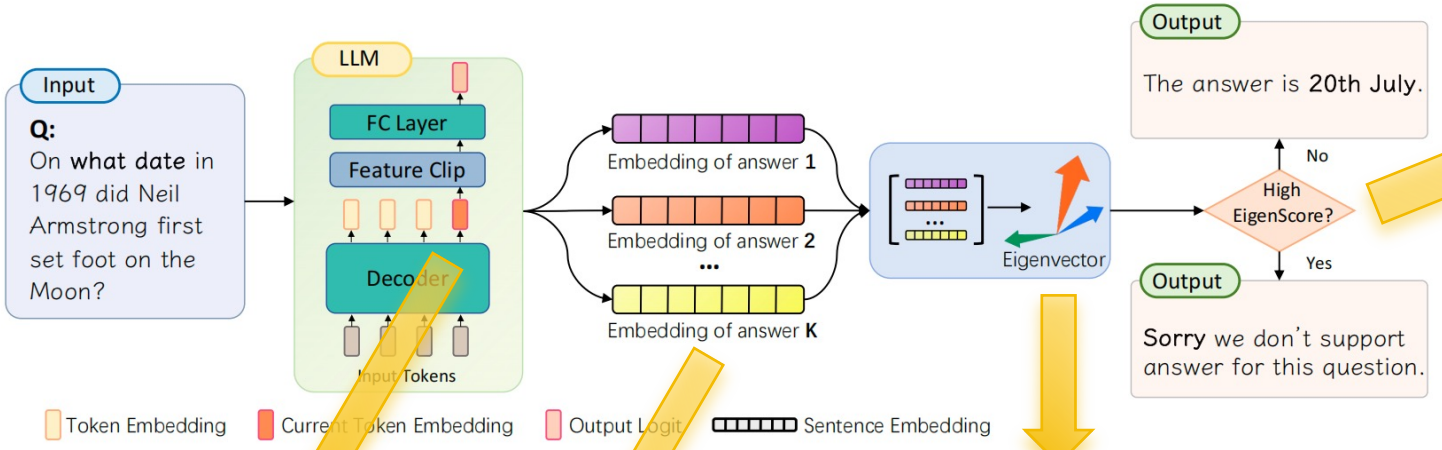
- **Introduction:** LofreeCP builds upon the CP method by transforming the computation of softmax into the computation of frequency, semantic entropy, and semantic similarity, thus providing a computation method for black-box models that does not require obtaining logits.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Consensus-based	Decoding	Black-box	Post-hoc	Medium	Coverage/Ranking

[91] API Is Enough: Conformal Prediction for Large Language Models Without Logit-Access, EMNLP 2024

EigenScore

EigenScore[13]



Sentence Embedding Extraction

INSIDE takes the last-token **hidden state from a middle layer** as the sentence embedding z:

$$z = h_T$$

1. Covariance Matrix of K Sampled Embeddings

Given K generated answer embeddings:

$$Z = [z_1, z_2, \ldots, z_K] \in \mathbb{R}^{d \times K}$$

Covariance matrix:

$$\Sigma = Z^\top \cdot J_d \cdot Z$$

where

- $J_d = I_d - \frac{1}{d} 1_d 1_d^\top$ (centering matrix)

2. EigenScore Definition (LogDet of Covariance)

$$E(\mathcal{Y}|x,\theta) = \frac{1}{K} \log\det(\Sigma + \alpha I_K)$$

- αI_K: small regularization to make the covariance matrix full-rank

3. Equivalent Eigenvalue Expression

$$E(\mathcal{Y}|x,\theta) = \frac{1}{K} \sum_{i=1}^{K} \log(\lambda_i)$$

Conceptual Meaning

- **Large EigenScore** → embeddings spread out → *high uncertainty*
- **Small EigenScore** → embeddings clustered → *model is confident*

[13] INSIDE: LLMs' Internal States Retain the Power of Hallucination Detection, ICLR 2024

EigenScore[13]

- **Task Type:** Open-ended generation

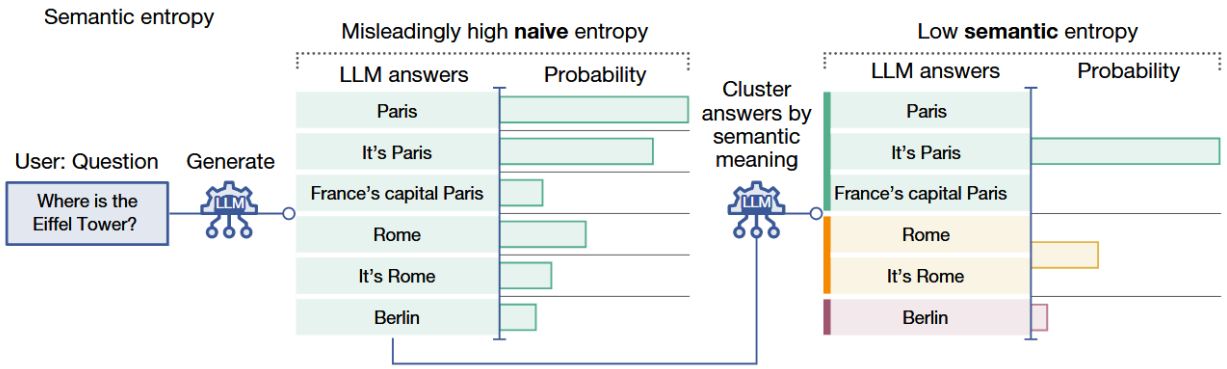
- **Introduction:** EigenScore constructs a representation-level uncertainty estimator by extracting last-token hidden states from a middle transformer layer and computing an EigenScore derived from the log-determinant of their covariance matrix across sampled generations. It quantifies decoding uncertainty by measuring divergence in the model's internal semantic representations, capturing how consistently the model encodes its own answers.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Consensus-based	Decoding	White-box	Post-hoc	Low	Ranking

[13] INSIDE: LLMs' Internal States Retain the Power of Hallucination Detection, ICLR 2024

Semantic Entropy

Sematic Entropy[32]



Algorithm 1 Bidirectional Entailment Clustering

Require: context x, set of seqs. $\{\mathbf{s}^{(2)}, \ldots, \mathbf{s}^{(M)}\}$, NLI classifier $\mathcal{M}$, set of meanings $C = \{\{\mathbf{s}^{(1)}\}\}$
for $2 \leq m \leq M$ **do**
  for $c \in C$ **do** ▷ Compare to already-processed meanings.
    $\mathbf{s}^{(c)} \leftarrow c_0$ ▷ Use first sequence for each semantic-class.
    `left` $\leftarrow \mathcal{M}(\text{cat}(x, \mathbf{s}^{(c)}, \text{“<g/>”}, x, \mathbf{s}^{(m)}))$ ▷ Does old sequence entail new one?
    `right` $\leftarrow \mathcal{M}(\text{cat}(x, \mathbf{s}^{(m)}, \text{“<g/>”}, x, \mathbf{s}^{(c)}))$ ▷ Vice versa?
    if `left` is `entailment` **and** `right` is `entailment` **then**
      $c \leftarrow c \bigcup \mathbf{s}^{(m)}$ ▷ Put into existing class.
    end if
  end for
  $C \leftarrow C \bigcup \{\mathbf{s}^{(m)}\}$ ▷ Semantically distinct, gets own class.
end for
return C

Main Steps:

1. **Generation:** Sample **M** sequences {s(1), . . . , s(M)} from the predictive distribution.
2. **Clustering:** Cluster the sequences which mean the same thing. (use a Deberta-large[45] model that is fine-tuned on the NLI data set MNLI[46].)
3. **Entropy estimation:** Approximate semantic entropy by summing probabilities and compute resulting entropy.

$$SE(x) = -\sum_c p(c \mid x) \log p(c \mid x)$$

$$= -\sum_c \left(\left(\sum_{\mathbf{s} \in c} p(\mathbf{s} \mid x) \right) \log \left[\sum_{\mathbf{s} \in c} p(\mathbf{s} \mid x) \right] \right).$$

(a) Scenario 1: No semantic equivalence

Answer s	Likelihood $p(\mathbf{s} \mid x)$	Semantic likelihood $\sum_{\mathbf{s} \in c} p(\mathbf{s} \mid x)$
Paris	0.5	0.5
Rome	0.4	0.4
London	0.1	0.1
Entropy	0.94	0.94

(b) Scenario 2: Some semantic equivalence

Answer s	Likelihood $p(\mathbf{s} \mid x)$	Semantic likelihood $\sum_{\mathbf{s} \in c} p(\mathbf{s} \mid x)$
Paris	0.5	0.9
It's Paris	0.4	
London	0.1	0.1
Entropy	0.94	0.33

[32] Detecting hallucinations in large language models using semantic entropy, Nature 2024

Sematic Entropy[32]

- **Task Type:** Open-ended/Math generation

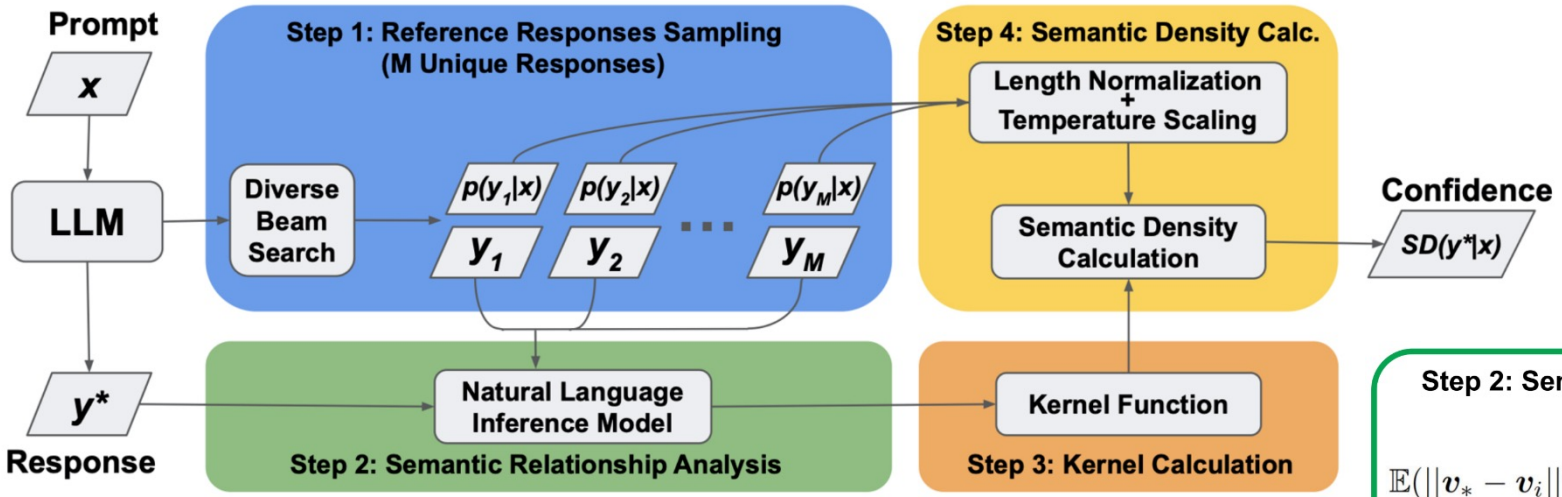
- **Introduction:** Semantic Entropy measures LLM uncertainty in semantic space, by quantifying how many distinct meanings its free-form outputs can take. It samples multiple generations, groups them into meaning-equivalent clusters using NLI-based entailment, and computes entropy over the cluster probabilities. Semantic Entropy predicts answer correctness more reliably than lexical entropy and self-evaluation.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Consensus-based	Decoding/Token-level	White-box	Post-hoc	Medium	Ranking

[32] Detecting hallucinations in large language models using semantic entropy, Nature 2024

Semantic Density

Semantic Density[79]



Algorithm 1 Procedure for deploying semantic density

Require:

$\boldsymbol{y}_*$: target response that needs confidence measurement

$\boldsymbol{x}$: original prompt for generating $\boldsymbol{y}_*$

M: number of unique reference responses to be sampled given $\boldsymbol{x}$

Ensure:

$\mathrm{SD}(\boldsymbol{y}_*|\boldsymbol{x})$: semantic density for $\boldsymbol{y}_*$ given $\boldsymbol{x}$

Step 1: Reference Response Sampling:

1: sample M unique reference responses $\boldsymbol{y}_i$ (for $i = 1, 2, \cdots, M$) with prompt $\boldsymbol{x}$ on the original LLM using diverse beam search, and record each corresponding length-normalized sampling probability $\sqrt[L_i]{p(\boldsymbol{y}_i|\boldsymbol{x})}$

Step 2: Semantic Relationship Analysis:

2: **for** $i = 1$ to M **do**

3: obtain $p_{\mathrm{c}}(\boldsymbol{y}_*, \boldsymbol{y}_i|\boldsymbol{x})$ and $p_{\mathrm{n}}(\boldsymbol{y}_*, \boldsymbol{y}_i|\boldsymbol{x})$ using NLI classification model

4: calculate expectation $\mathbb{E}(||\boldsymbol{v}_* - \boldsymbol{v}_i||^2) = p_{\mathrm{c}}(\boldsymbol{y}_*, \boldsymbol{y}_i|\boldsymbol{x}) + \frac{1}{2} \cdot p_{\mathrm{n}}(\boldsymbol{y}_*, \boldsymbol{y}_i|\boldsymbol{x})$

Step 3: Kernel Function Calculation:

5: **for** $i = 1$ to M **do**

6: calculate kernel function value using the expectation of $||\boldsymbol{v}_* - \boldsymbol{v}_i||^2$, given by:

$K(\boldsymbol{v}_* - \boldsymbol{v}_i) = (1 - \mathbb{E}(||\boldsymbol{v}_* - \boldsymbol{v}_i||^2))\mathbf{1}_{\mathbb{E}(||\boldsymbol{v}_* - \boldsymbol{v}_i||) \leq 1}$

Step 4: Semantic Density Calculation:

7: calculate semantic density: $\mathrm{SD}(\boldsymbol{y}_*|\boldsymbol{x}) = \frac{1}{\sum_{i=1}^{M} \sqrt[L_i]{p(\boldsymbol{y}_i|\boldsymbol{x})}} \sum_{i=1}^{M} \sqrt[L_i]{p(\boldsymbol{y}_i|\boldsymbol{x})} K(\boldsymbol{v}_* - \boldsymbol{v}_i)$

Step 2: Semantic Relationship Analysis

Contradictory Prob. Neutral Prob. Entailment Prob.

$$\mathbb{E}(||\boldsymbol{v}_* - \boldsymbol{v}_i||^2) = 1^2 \cdot p_{\mathrm{c}}(\boldsymbol{y}_*, \boldsymbol{y}_i|\boldsymbol{x}) + (\tfrac{\sqrt{2}}{2})^2 \cdot p_{\mathrm{n}}(\boldsymbol{y}_*, \boldsymbol{y}_i|\boldsymbol{x}) + 0^2 \cdot p_{\mathrm{e}}(\boldsymbol{y}_*, \boldsymbol{y}_i|\boldsymbol{x})$$
$$= p_{\mathrm{c}}(\boldsymbol{y}_*, \boldsymbol{y}_i|\boldsymbol{x}) + \tfrac{1}{2} \cdot p_{\mathrm{n}}(\boldsymbol{y}_*, \boldsymbol{y}_i|\boldsymbol{x})$$

Step 3: Kernel Calculation

$$K(\boldsymbol{v}_* - \boldsymbol{v}_i) = (1 - ||\boldsymbol{v}_* - \boldsymbol{v}_i||^2)\mathbf{1}_{||\boldsymbol{v}_* - \boldsymbol{v}_i|| \leq 1}$$

Estimated by $\mathbb{E}(||\boldsymbol{v}_* - \boldsymbol{v}_i||^2)$

Step 4: Semantic Density Calc.

$$\mathrm{SD}(\boldsymbol{y}_*|\boldsymbol{x}) = \frac{1}{\sum_{i=1}^{M} p(\boldsymbol{y}_i|\boldsymbol{x})} \sum_{i=1}^{M} p(\boldsymbol{y}_i|\boldsymbol{x}) K(\boldsymbol{v}_* - \boldsymbol{v}_i)$$

Post-processing by Length-normalization + Temperature Scaling

Sampling Probability by LLM

[79] Semantic Density: Uncertainty Quantification for Large Language Models through Confidence Measurement in Semantic Space, Neurips 2024

Semantic Density[79]

- **Task Type:** Open-ended generation

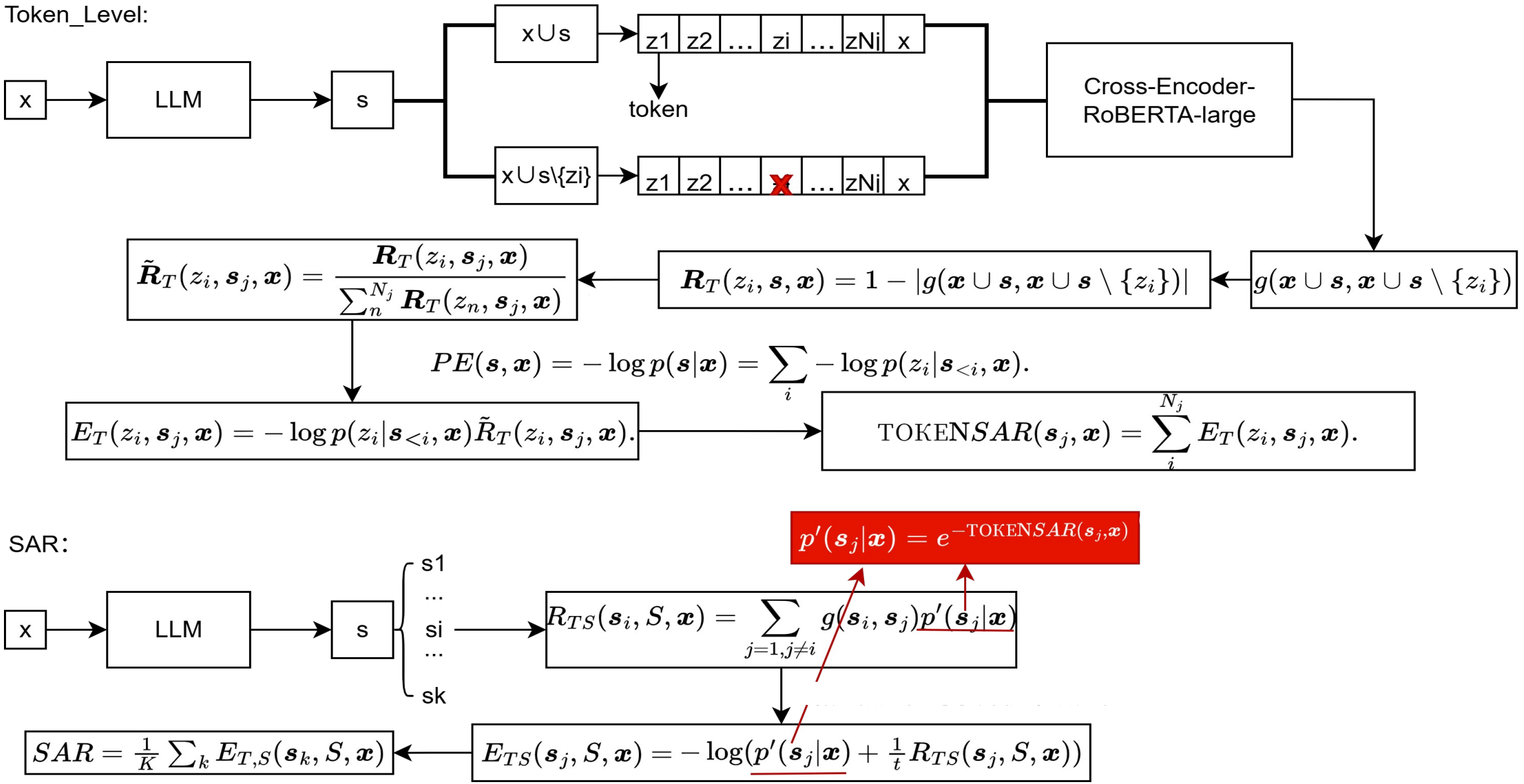
- **Introduction:** Semantic Density measures LLM uncertainty by evaluating how tightly plausible alternative responses cluster around the primary answer in semantic space. It generates a main answer and diverse references, analyzes their relations via NLI, and computes a density score reflecting reasoning consistency. Semantic Density provides an interpretable uncertainty signal complementary to probability-based metrics.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Consensus-based	Decoding/Token-level	White-box	Post-hoc	Medium	Ranking

[79] Semantic Density: Uncertainty Quantification for Large Language Models through Confidence Measurement in Semantic Space, Neurips 2024

SAR

SAR[29]



[29] Shifting Attention to Relevance: Towards the Predictive Uncertainty Quantification of Free-Form Large Language Models, ACL 2024

SAR[29]

- **Task Type:** Open-ended generation

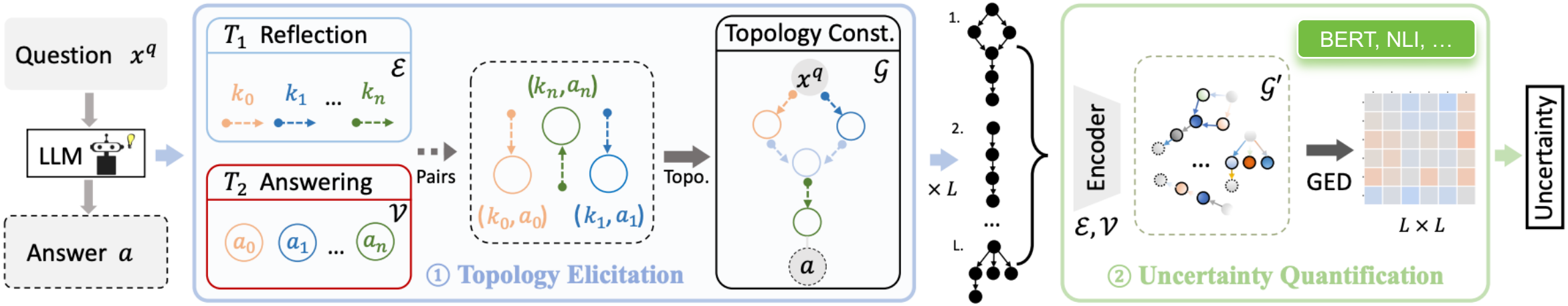
- **Introduction:** SAR reweights token-level and sentence-level predictive entropies using relevance scores to refine uncertainty estimates. It quantifies decoding/token-level uncertainty and provides a structured post-hoc adjustment that emphasizes semantically meaningful components.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Consensus-based	Decoding/Token-level	White-box	Post-hoc	Medium	Ranking

[29] Shifting Attention to Relevance: Towards the Predictive Uncertainty Quantification of Free-Form Large Language Models, ACL 2024

Topo-UQ

Topo-UQ[22]



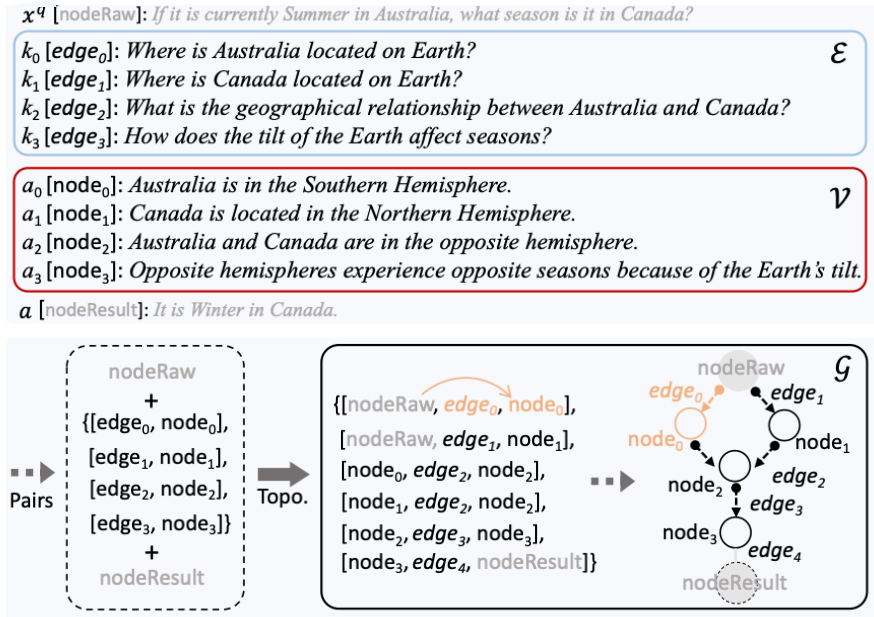
☐ Step 1: Topology Elicitation

Given an input question x^q, the model first generates sub-questions k_n, and then produces corresponding answers, forming sub-question–sub-answer pairs (k_n, a_n), where sub-questions and sub-answers are respectively treated as edges and nodes; the model further outputs dependency structures of the form $[node_i, edge_{ij}, node_j]$, organizing these steps into a reasoning topology G.

☐ Step 2: Uncertainty Quantification

given multiple reasoning topologies for the same query, the model encodes nodes and edges into semantic embeddings and computes pairwise differences using a graph edit distance variant, where uncertainty is defined as the variance of these distances, capturing both semantic and structural inconsistency.

e.g.,



[22] Understanding the Uncertainty of LLM Explanations: A Perspective Based on Reasoning Topology, COLM 2025

Topo-UQ[22]

- **Task Type:** Opening-end/Math/Code generation

- **Introduction:** Topo-UQ constructs reasoning topologies as graphs and computes uncertainty via semantic-aware graph edit distances across multiple explanations, representing reasoning through structured graph forms. It quantifies reasoning uncertainty and provides a structured perspective for analyzing consistency and variance in reasoning processes.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Consensus-based	Reasoning	Black-box	Post-hoc	Medium	Ranking

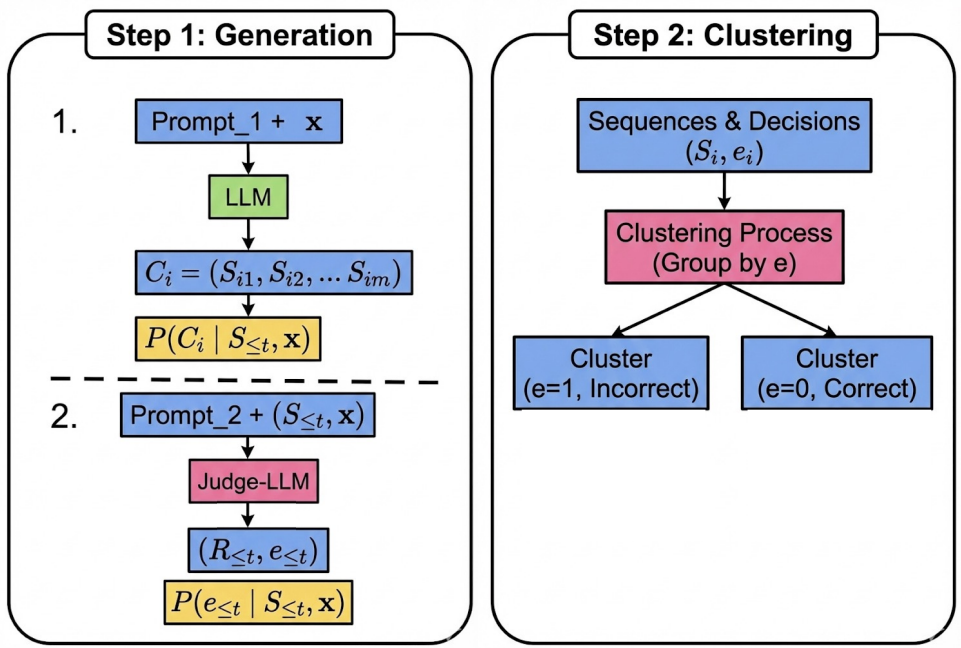
[22] Understanding the Uncertainty of LLM Explanations: A Perspective Based on Reasoning Topology, COLM 2025

CoT Entropy

CoT Entropy[119]

Main Steps:

1. **Generation:** Given a judge-LM and a context $\mathbf{x}_{\leq t}$, sample reasoning sequences c that critique the current step s_t, followed by a decision e (of whether s_t is deemed incorrect) after the CoT.
2. **Clustering:** Group the output sequences into two clusters based on whether the decision e equals 1 (deemed incorrect) or 0 (deemed correct).
3. **Entropy estimation:** Normalize token probabilities $\tau(e)$ to obtain class probabilities for each sequence in both clusters. Then, sum the class probabilities of sequences leading to the same decision e and compute the resulting entropy.



Step 3: Entropy estimation

$$\mathbf{x}_{\leq t} \coloneqq (Q, \mathbf{s}_{\leq t}, \mathbf{I}_{\text{CoT}}),$$

$$\text{CoTE}(\mathbf{x}_{\leq t}) = -\sum_e \left(\left[\sum_c p_\theta(c, e \mid \mathbf{x}_{\leq t}) \right] \log \left[\sum_c p_\theta(c, e \mid \mathbf{x}_{\leq t}) \right] \right)$$

$$= -\sum_e \left(\left[\sum_c p_\theta(e \mid \mathbf{x}_{\leq t}, c) p_\theta(c \mid \mathbf{x}_{\leq t}) \right] \log \left[\sum_c p_\theta(e \mid \mathbf{x}_{\leq t}, c) p_\theta(c \mid \mathbf{x}_{\leq t}) \right] \right),$$

[119] Uncertainty-aware Step-wise Verification with Generative Reward Models, arxiv 2025

CoT Entropy[119]

- **Task Type:** Opening-end/Math/Code generation

- **Introduction:** CoT Entropy, extending semantic entropy, marginalizes chain-of-thought rationales to estimate posterior entropy over step-wise verification decisions. CoT Entropy quantifies reasoning uncertainty, providing a structured probabilistic signal for uncertainty-aware process reward modeling.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Consensus-based	Reasoning	White-box	Post-hoc	Low	Ranking

[119] Uncertainty-aware Step-wise Verification with Generative Reward Models, arxiv 2025

Single-pass Methods

Methods	Papers
CP-MCQA	Conformal Prediction with Large Language Models for Multi-Choice Question Answering, ICML 2023
MARS	MARS: Meaning-Aware Response Scoring for Uncertainty Estimation in Generative LLMs, ACL 2024
CCP	Fact-Checking the Output of Large Language Models via Token-Level Uncertainty Quantification, ACL 2024
SEPs	Semantic Entropy Probes: Robust and Cheap Hallucination Detection in LLMs, arxiv 2024
BLL	Can Linear Probes Measure LLM Uncertainty, Neurips Workshop 2025
LACIE	LACIE: Listener-Aware Finetuning for Confidence Calibration in Large Language Models, Neurips 2024
SaySelf	SaySelf: Teaching LLMs to Express Confidence with Self-Reflective Rationales, arxiv 2024
Linguistic Calibration	Linguistic Calibration of Long-Form Generations, ICML 2024
CoT-UQ	CoT-UQ: Improving Response-wise Uncertainty Quantification in LLMs with Chain-of-Thought, ACL 2025
Verbalized Probability	Teaching Models to Express Their Uncertainty in Words, arxiv 2022
P(IK)	Language Models (Mostly) Know What They Know, arxiv 2022
P(True)	Language Models (Mostly) Know What They Know, arxiv 2022

CP-MCQA

CP-MCQA[51]

Steps:

50% of the data

Calibration Set

$S(x, Y) = 1 - [f(x)]_y$ ⇒ A classifier outputs softmax probability for each class

Calculate each sample's score

$S_1, S_2, \ldots, S_n$ (using the score of the ground-truth class)

Sort from small to large

α means the true label has at least 1−α probability of appearing in the prediction set C(x). Set to 0.1 in the paper.

$$\hat{q}_\alpha = \text{Quantile}\left(\{S_1, \ldots, S_n\}, \frac{(n+1)(n-\alpha)}{n}\right)$$

During inference

$$C(x) = y : S(x, y) \leq \hat{q}_a$$

$|C(x)|$ represents uncertainty

Example:
In Calibration Set

S_1	S_2	S_3	S_4	S_5
0.1	0.4	0.25	0.5	0.45

$\hat{q}_{0.1} \approx 0.45$

C(x) = {Cat}
|C(x)|=1 indicates low uncertainty

Class	$f(x)_y$	$S(x, y)$
Cat	0.6	0.4
Dog	0.3	0.7
Bird	0.1	0.9

[51] Conformal Prediction with Large Language Models for Multi-Choice Question Answering, ICML 2023

CP-MCQA[51]

- **Task Type:** Multiple-choice QA

- **Introduction:** Conformal Prediction for Multi-Choice Question Answering (CP-MCQA) filters out a set of predicted answers by calculating a softmax score, ensuring that the true answer lies in this set with a user-specified probability of $1-\alpha$. It quantifies the uncertainty of the prediction by the size of the prediction set.

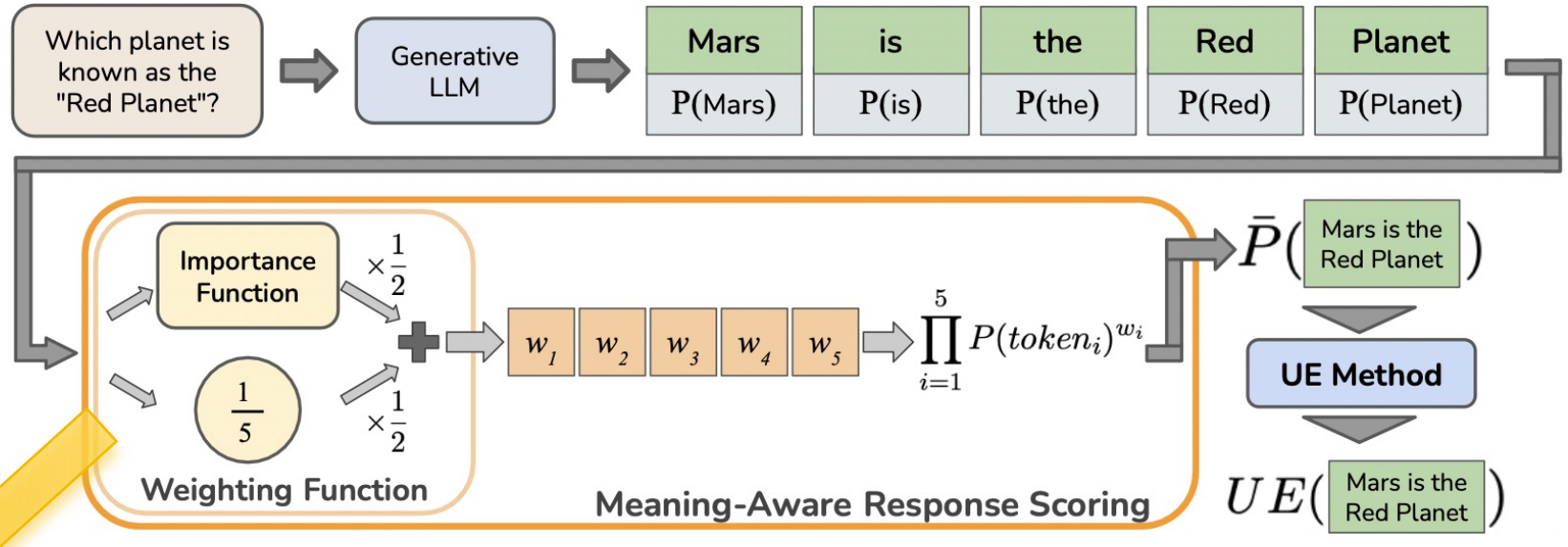
Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Single-pass	Token-level	White-box	Post-hoc	Low	Coverage/Ranking

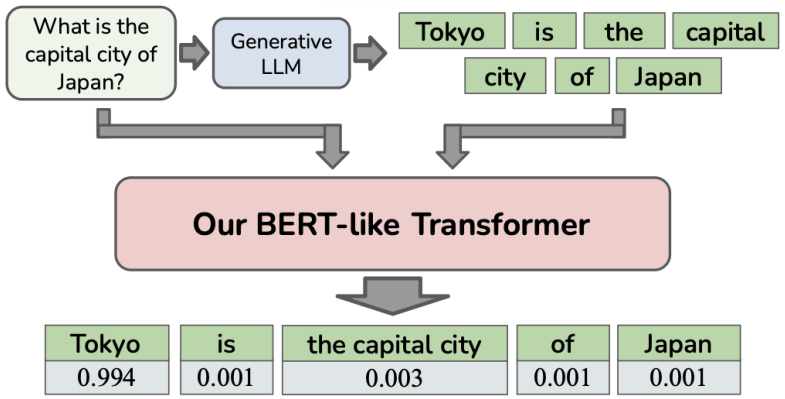
[51] Conformal Prediction with Large Language Models for Multi-Choice Question Answering, ICML 2023

MARS

MARS[6]

change token-level probabilities into importance weights reflecting how much each token contributes to meaning.





$$w(\mathbf{s}, \mathbf{x}, L, l) \triangleq \frac{1}{2L} + \frac{u(\mathbf{s}, \mathbf{x}, l)}{2}$$ ✅

each word's contribution to the sentence's meaning in the question context might vary

$$\tilde{P}(\mathbf{s}|\mathbf{x}, \theta) = \prod_{l=1}^{L} P(s_l|s_{<l}, \mathbf{x}; \theta)^{\frac{1}{L}}$$ ❌

[6] MARS: Meaning-Aware Response Scoring for Uncertainty Estimation in Generative LLMs, ACL 2024

MARS[6]

- **Task Type:** Open-ended generation

- **Introduction:** MARS reflects each token's semantic contribution to a generated answer, which replaces length-normalized scoring by weighting token probabilities accordingly. It quantifies prediction uncertainty and offers a meaning-aware scoring method for token-level uncertainty estimation in generative LLMs.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Single-pass	Token-level	White-box	Post-hoc	Medium	Ranking

[6] MARS: Meaning-Aware Response Scoring for Uncertainty Estimation in Generative LLMs, ACL 2024

CCP

CCP[30]

Core Idea:

- $CCP(x_j) = P\big(\text{Meaning}(x_{1:j}) | x_{<j}, \text{ClaimType}(x_{1:j})\big).$

$$= \frac{P(\text{Meaning}(x_{1:j}), \text{ClaimType}(x_{1:j}) \mid x_{<j})}{P(\text{ClaimType}(x_{1:j}) \mid x_{<j})}. \longrightarrow P(Meaning(x_{1:j})|x_{<j})$$

- $P(Meaning(x_{1:j})|x_{<j}) = \sum_{x_j^k \in M(x_j)} P(x_j^k|x_{<j})$
- $P(ClaimType(x_{1:j})) = \sum_{x_j^l \in CT(x_j)} P(x_j^l|x_{<j})$
- $CCP(x_j) = \frac{\sum_{x_j^k \in M(x_j)} P(x_j^k \mid x_{<j})}{\sum_{x_j^l \in CT(x_j)} P(x_j^l \mid x_{<j})}.$
- $CCP_{claim}(C) = 1 - \prod_{j \in C} CCP_{word}(x_j).$

Task：Fact-Checking

Sentence:
She attended the School of Design, where she earned a Bachelor of Fine Arts Degree in painting.

Fact: She earned a Bachelor of Fine Arts Degree in painting.

She attended the School of Design, where she earned a Bachelor of Fine Arts Degree in ...

- painting 49%
- art 3%
- acting 7%
- sculpture 2%
- 1977 39%

NLI(Bachelor of Fine Arts Degree in painting, Bachelor of Fine Arts Degree in painting) = entail
NLI(Bachelor of Fine Arts Degree in painting, Bachelor of Fine Arts Degree in art) = entail
NLI(Bachelor of Fine Arts Degree in painting, Bachelor of Fine Arts Degree in acting) = contra
NLI(Bachelor of Fine Arts Degree in painting, Bachelor of Fine Arts Degree in sculpture) = contra
NLI(Bachelor of Fine Arts Degree in painting, Bachelor of Fine Arts Degree in 1977) = neutral

top-K words (K=10)

CCP_{word}(painting) = (P(painting) + P(art)) / (P(painting) + P(art) + P(acting) + P(sculpture))
= 0.52 / 0.61 = 0.85

CCP_{word}(of) = CCP_{word}(in) = 1 (functional words)

CCP_{claim}(She earned a Bachelor of Fine Arts Degree in painting.) =
-(CCP_{word}(Bachelor) * CCP_{word}(of) * CCP_{word}(Fine) * CCP_{word}(Arts) * CCP_{word}(Degree) * CCP_{word}(in) * CCP_{word}(painting))

[30] Fact-Checking the Output of Large Language Models via Token-Level Uncertainty Quantification, ACL 2024

CCP[30]

- **Task Type:** Open-ended generation

- **Introduction:** CCP re-aggregates token logits based on semantics and claim type. It obtains a word-level CCP score by dividing the sum of probabilities of synonyms by the sum of probabilities of words of the same type. Then, it calculates the claim-level score using the token-level score. This method makes fact-checking independent of changes in surface word form or sentence structure.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Single-pass	Token-level	White-box	Post-hoc	Medium	Ranking

[30] Fact-Checking the Output of Large Language Models via Token-Level Uncertainty Quantification, ACL 2024

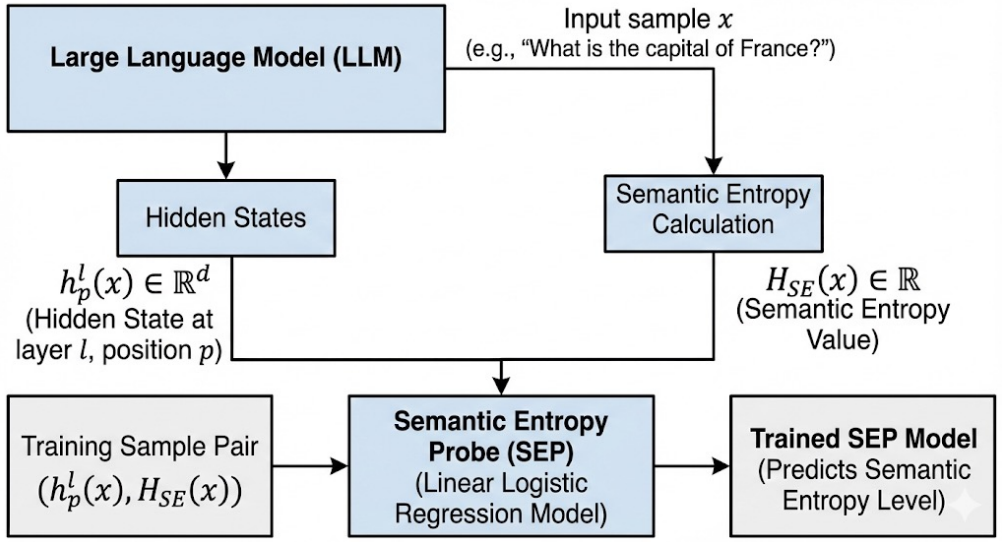
SEPs

SEPs[50]

Training SEPs

Semantic Entropy Probes (SEPs) are linear logistic regression models trained on LLM hidden states to predict semantic entropy.

Each training sample pairs a hidden representation $h_p^l(x) \in \mathbb{R}^d$ with its corresponding semantic entropy $H_{SE}(x) \in \mathbb{R}$ for input x.



Binarization

The semantic entropy $H_{SE}(x)$ is a continuous value and is binarized to enable classification into high- and low-entropy groups:

$$\tilde{H}_{SE}(x) = \mathbf{1}\left[H_{SE}(x) > \gamma^*\right].$$

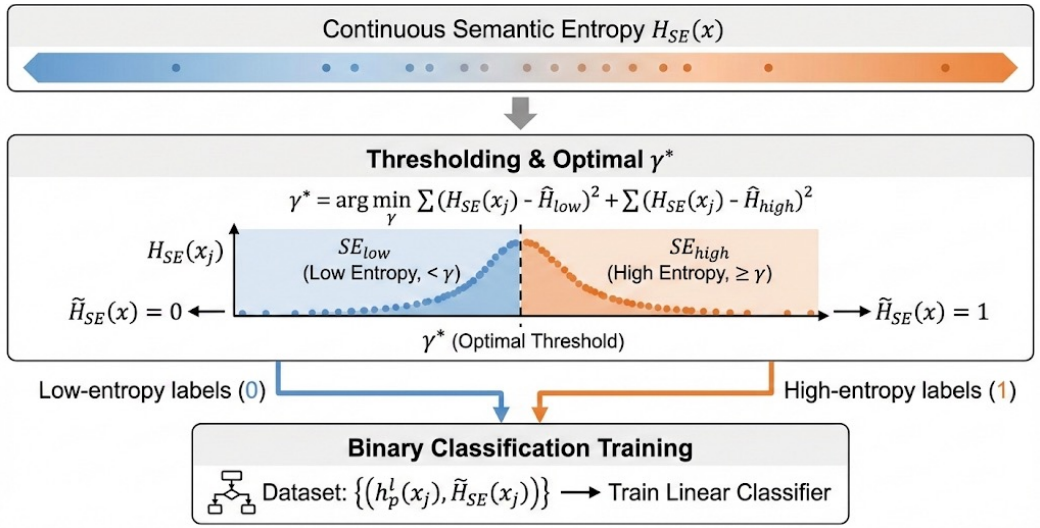
The threshold is chosen to minimize within-group variance:

$$\gamma^* = \arg\min_{\gamma} \sum_{j \in SE_{\text{low}}} \left(H_{SE}(x_j) - \hat{H}_{\text{low}}\right)^2 + \sum_{j \in SE_{\text{high}}} \left(H_{SE}(x_j) - \hat{H}_{\text{high}}\right)^2,$$

where

$$SE_{\text{low}} = \{j : H_{SE}(x_j) < \gamma\}, \qquad SE_{\text{high}} = \{j : H_{SE}(x_j) \geq \gamma\}.$$

The resulting dataset $\left\{\left(h_p^l, (x_j)\tilde{H}_{SE}(x_j)\right)\right\}$ is used to train a linear classifier for predicting high semantic entropy.



[50] Semantic Entropy Probes: Robust and Cheap Hallucination Detection in LLMs, arxiv 2024

SEPs[50]

- **Task Type:** Open-ended generation

- **Introduction:** Semantic Entropy Probes (SEPs) train a lightweight linear model on LLM hidden states to approximate semantic entropy. The probe is trained with hidden-state/semantic-entropy pairs supervision and outputs uncertainty estimates via a single forward pass at inference. SEPs provides an efficient and scalable semantic-level uncertainty estimator while avoiding the computational cost of sampling-based entropy measures.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Single-pass	Decoding	White-box	Fine-tuning	High	Ranking

[50] Semantic Entropy Probes: Robust and Cheap Hallucination Detection in LLMs, arxiv 2024

BLL

BLL[23]

1. Collect Layer Activations

- Query LLM on training questions
- Extract layer activations $h^{(l,i)}$
- Compute activation change:

$$y^{(l,i)} = h^{(l,i)} - h^{(l-1,i)}$$

- Split into **Cor** (correct) and **Incor** (incorrect)

2. Bayesian Linear Modeling (Per Neuron, Per Mode)

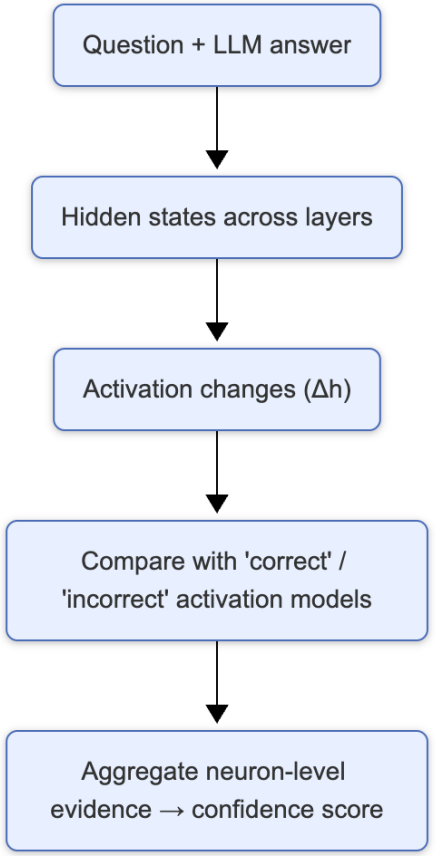
For each neuron (l, i), fit **two Bayesian linear models**:

$$y^{(l,i)} = w_{u,0}^{(l,i)} + w_u^{(l,i)}\, h^{(l-1,:)} + \varepsilon_u^{(l,i)}, \quad \varepsilon_u \sim \mathcal{N}(0, \sigma_u^2)$$

- $u = 1$: Correct mode
- $u = 0$: Incorrect mode
- Obtain posterior predictive:

$$p\left(y^{(l,i)} \mid h^{(l-1,:)}, u\right)$$

3. Posterior → Global Confide



BLL extracts two feature types:

(A) Posterior log-likelihood

$$-\log p\left(y^{(l,i)} \mid u\right)$$

(B) Likelihood Ratio (LLR)

$$\mathrm{LLR}^{(l,i)} = \log \frac{p\left(y^{(l,i)} \mid u = 1\right)}{p\left(y^{(l,i)} \mid u = 0\right)}$$

- Indicates whether the activation **resembles the correct or incorrect mode**
- All neuron-level features combined via **Elastic-Net probe** → final confidence score

[23] Can Linear Probes Measure LLM Uncertainty, Neurips Workshop 2025

BLL[23]

- **Task Type:** Open-ended generation

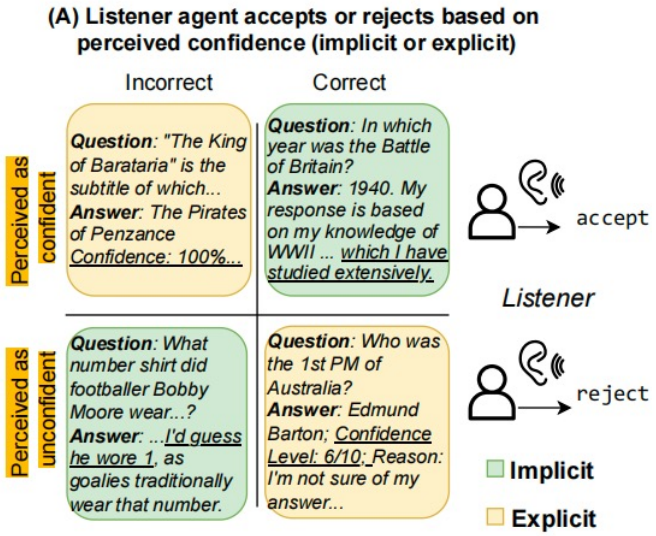
- **Introduction:** BLL fits Bayesian linear models to layer-wise activation differences conditioned on correctness and aggregates posterior features through sparse regression to estimate response reliability. It quantifies decoding uncertainty and provides a structured way to extract distributional signals from intermediate activations for factual uncertainty assessment.

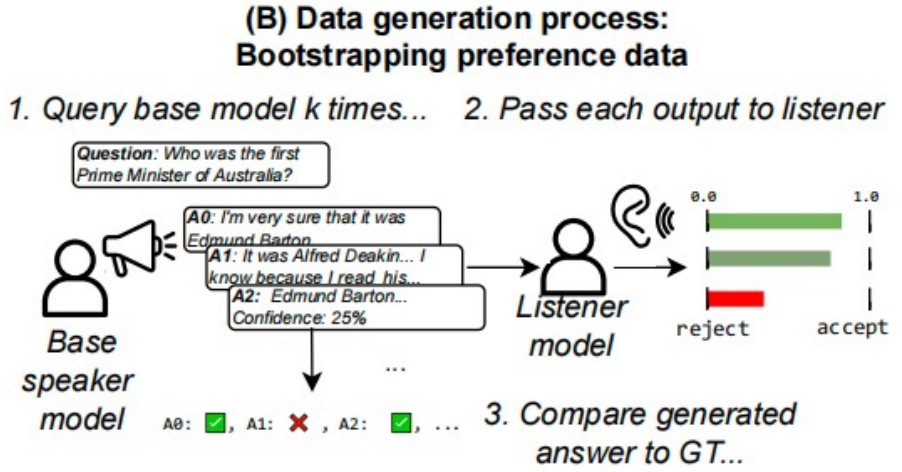
Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Single-pass	Decoding	White-box	Fine-tuning	High	Ranking/Calibration

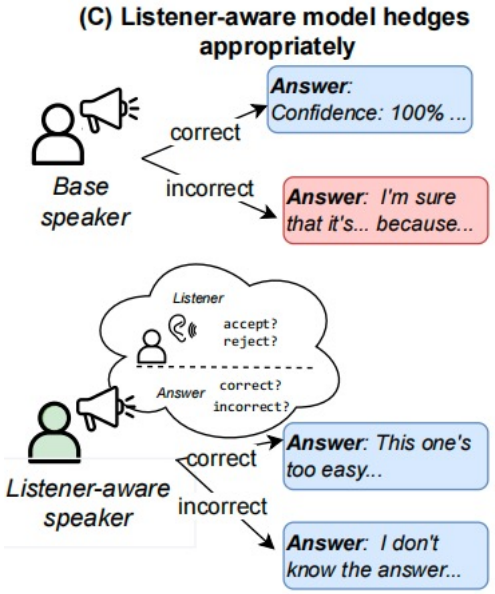
[23] Can Linear Probes Measure LLM Uncertainty, Neurips Workshop 2025

LACIE

LACIE[90]







be confident regardless of whether they are right or wrong

more confident when correct

less confident when wrong

1. For each training question, generate k diverse responses.
2. Then scored for correctness against the gold answers and accepted or rejected by a listener model.
3. The preference function rewards true accepts and true rejects and penalizes false accepts and false rejects. (DPO)

[90] LACIE: Listener-Aware Finetuning for Confidence Calibration in Large Language Models, Neurips 2024

LACIE[90]

- **Task Type:** Open-ended generation

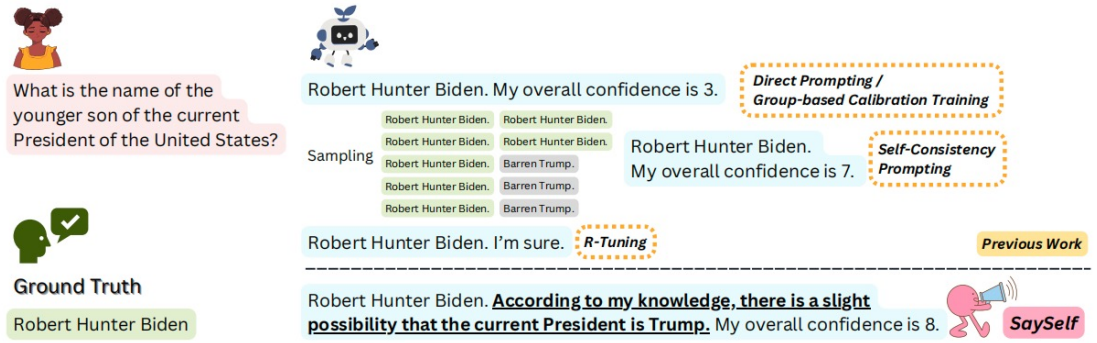
- **Introduction:** LACIE calibrates confidence by finetuning a speaker model using preference data generated through a multi-agent speaker–listener game that evaluates both correctness and perceived confidence. It quantifies decoding uncertainty and provides a pragmatic mechanism for aligning explicit and implicit confidence cues with how listeners interpret model outputs.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Single-pass	Decoding	White-box	Fine-tuning	High	Calibration/Ranking

[90] LACIE: Listener-Aware Finetuning for Confidence Calibration in Large Language Models, Neurips 2024

SaySelf

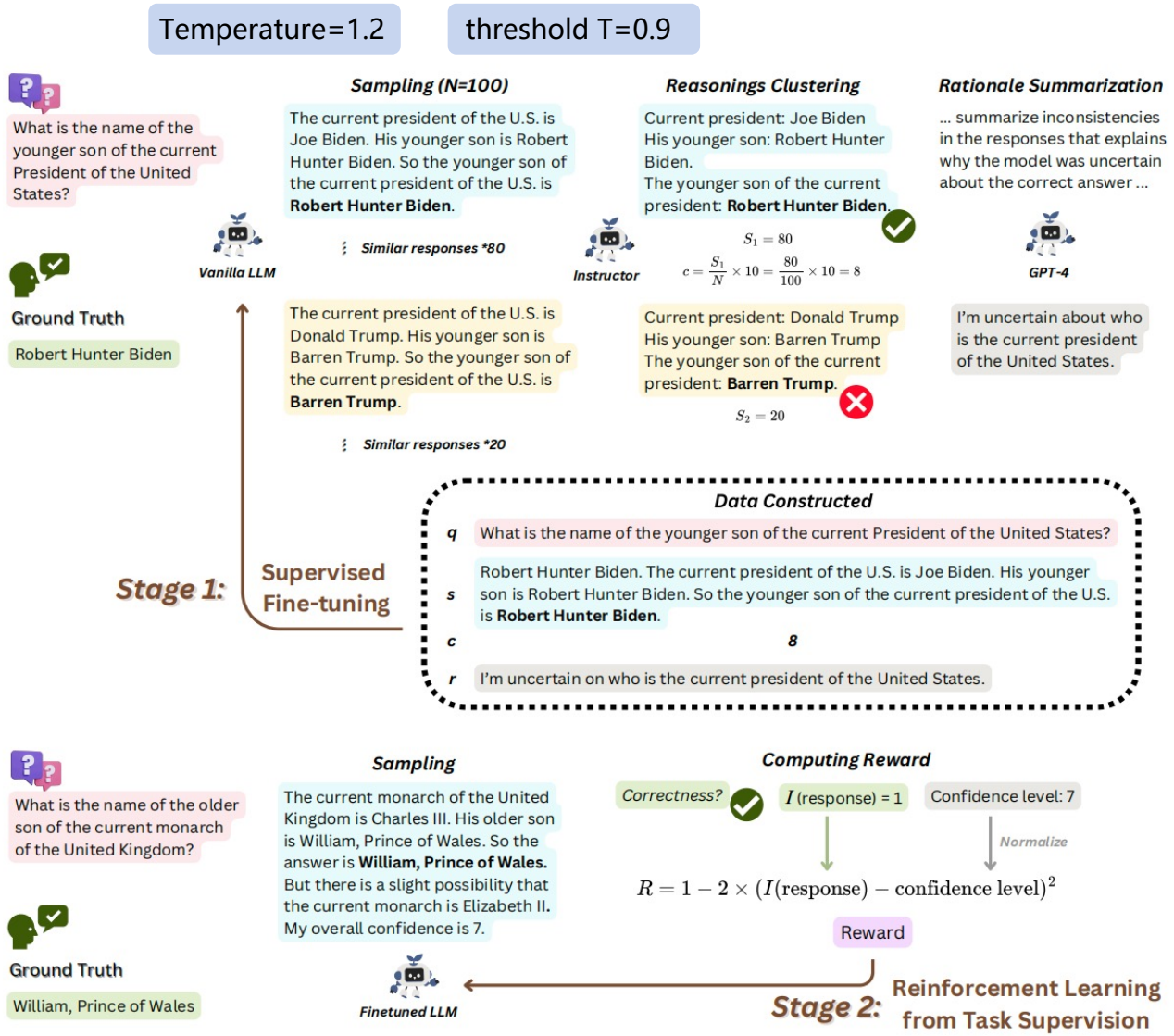
SaySelf[113]



☐ Overview

SaySelf trains LLMs in **two stages**:

- **Stage 1 – Supervised Fine-tuning :**
  1. LLM generates multiple answers for the same question and clusters them.
  2. GPT-4 summarizes uncertainty causes (the "self-reflective rationale").
  3. The model learns to output answer + rationale + confidence level.
- **Stage 2 – Reinforcement Learning :**
  1. Confidence values are refined to align predicted confidence with empirical correctness.



[113] SaySelf: Teaching LLMs to Express Confidence with Self-Reflective Rationales, arxiv 2024

SaySelf[113]

- **Task Type:** Open-ended generation

- **Introduction:** SaySelf introduces a self-reflective framework that enables LLMs to express their own confidence levels. Through a two-stage process involving SFT and PPO RL, the model learns to generate task responses, rationales and confidence scores, improving both confidence calibration and interpretability.

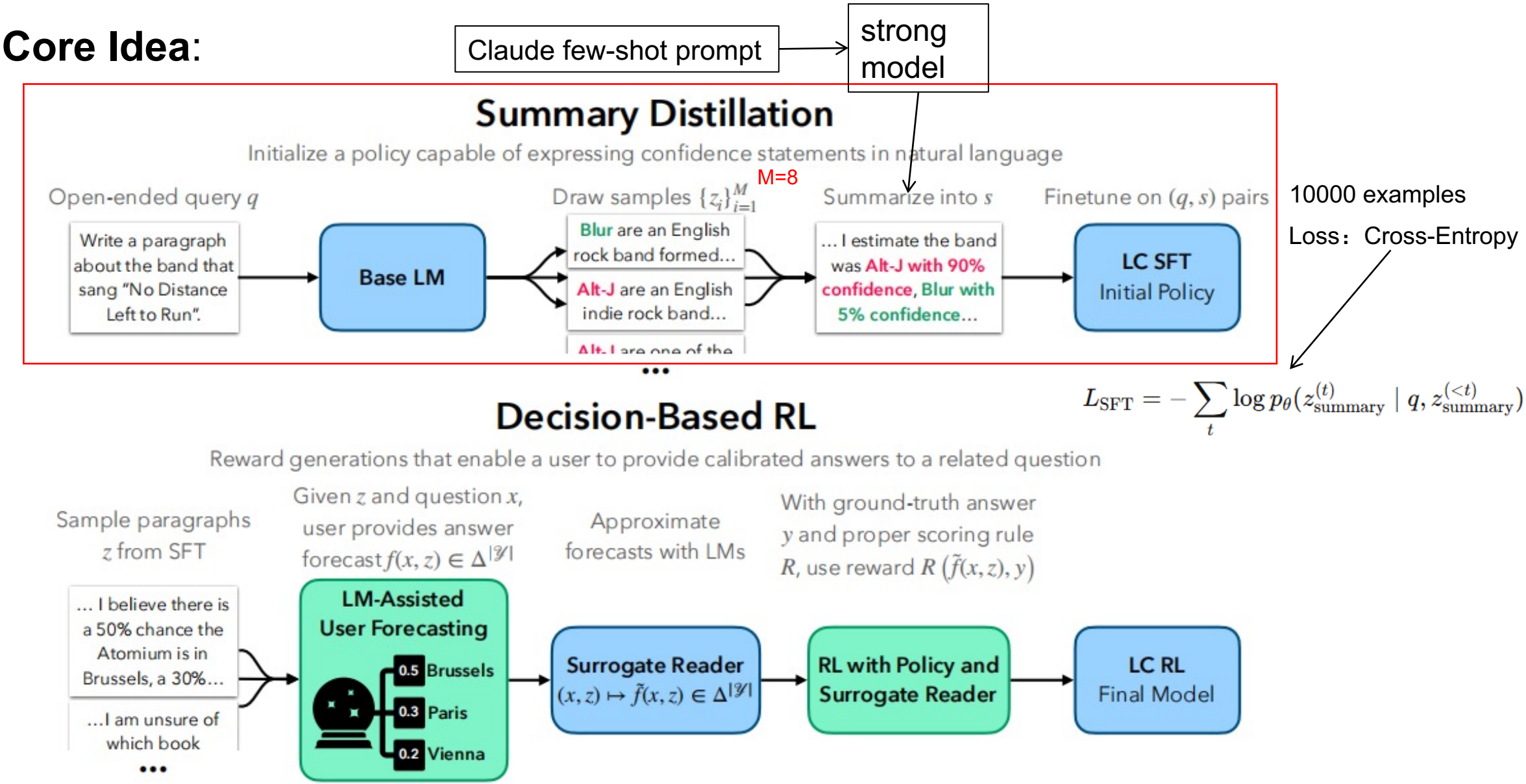
Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Single-pass	Reasoning	White-box	Fine-tuning	High	Calibration

[113] SaySelf: Teaching LLMs to Express Confidence with Self-Reflective Rationales, arxiv 2024

Linguistic Calibration

Linguistic Calibration[8]

Core Idea:



[8] Linguistic Calibration of Long-Form Generations, ICML 2024

Linguistic Calibration[8]

- **Task Type:** Open-ended generation

- **Introduction:** Linguistic Calibration adjusts LLM outputs by extracting linguistic cues of certainty—such as hedging and modality—and mapping them to calibrated confidence scores. It quantifies reasoning uncertainty and provides a structured post-hoc way to align verbal expressions with probabilistic confidence.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Single-pass	Reasoning	White-box	Fine-tuning	High	Calibration

[8] Linguistic Calibration of Long-Form Generations, ICML 2024

CoT-UQ

CoT-UQ[122]

☐ Step 1: Reasoning Extraction

The model generates an explicit step-by-step reasoning process using Chain-of-Thought prompting, together with a final answer.

☐ Step 2: Keywords Extraction

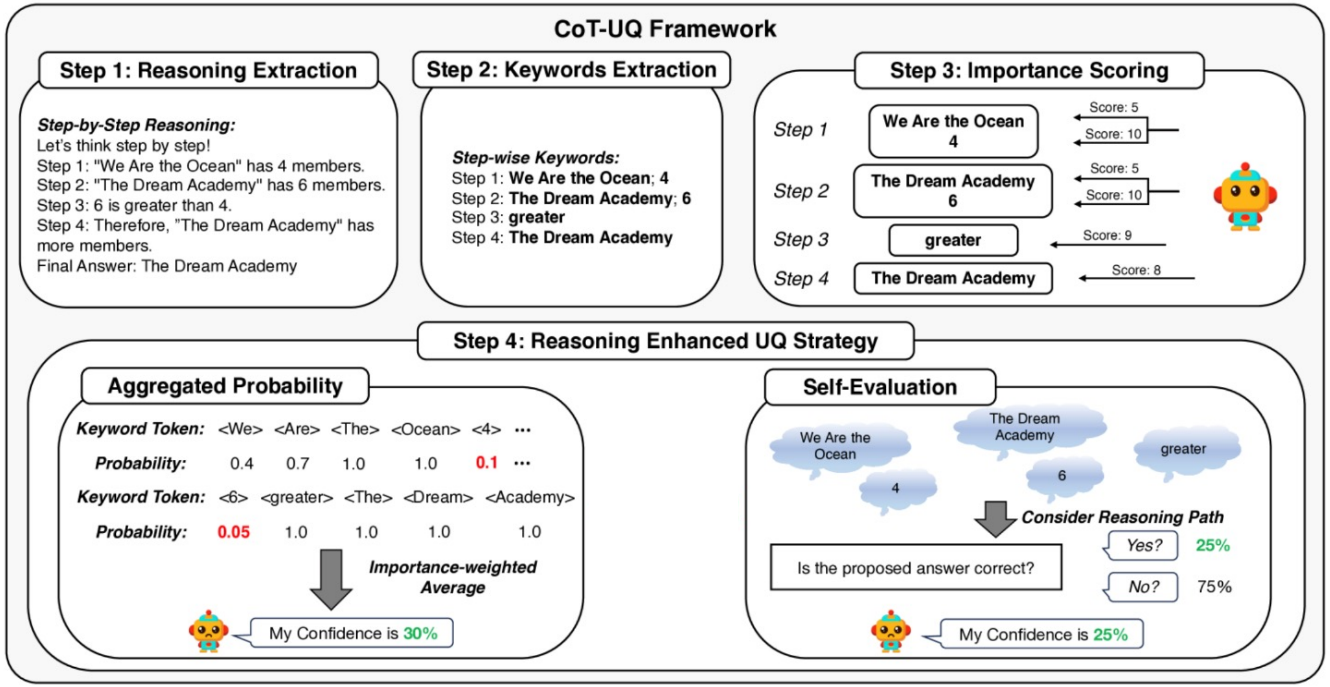
Key tokens (e.g., entities, numbers, logical relations) are extracted from each reasoning step to retain only the most informative reasoning components.

☐ Step 3: Importance Scoring

Each extracted keyword is assigned an importance score indicating its contribution to deriving the final answer.

☐ Step 4: Reasoning Enhanced UQ Strategy

The extracted keywords and their importance scores are integrated into existing uncertainty estimation strategies(AP & SE) to produce calibrated response-wise uncertainty.



[122] CoT-UQ: Improving Response-wise Uncertainty Quantification in LLMs with Chain-of-Thought, ACL 2025

CoT-UQ[122]

- **Task Type:** Opening-end/Math/Code generation

- **Introduction:** CoT-UQ integrates Chain-of-Thought reasoning into uncertainty quantification by extracting step-wise keywords, assigning importance scores, and aggregating their token-level probabilities to form a response-wise confidence score. It quantifies reasoning uncertainty and provides a structured way to incorporate internal reasoning signals into existing UQ strategies.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Single-pass	Reasoning/Token-level	White-box	Post-hoc	Medium	Ranking

[122] CoT-UQ: Improving Response-wise Uncertainty Quantification in LLMs with Chain-of-Thought, ACL 2025

Verbalized Probability

Verbalized Probability[58]

- **Pipeline:**
  - **Fine-tuning**

    Step 1: Sub-task Correctness Estimation

    Task: add/sub × 100(questions)

Prompt + $q_T \times 100$ → LLM → $a_M \times 100$

$$\hat{p}_T = \mathbb{E}_{q \in T}[\mathbb{I}(a_M)]$$

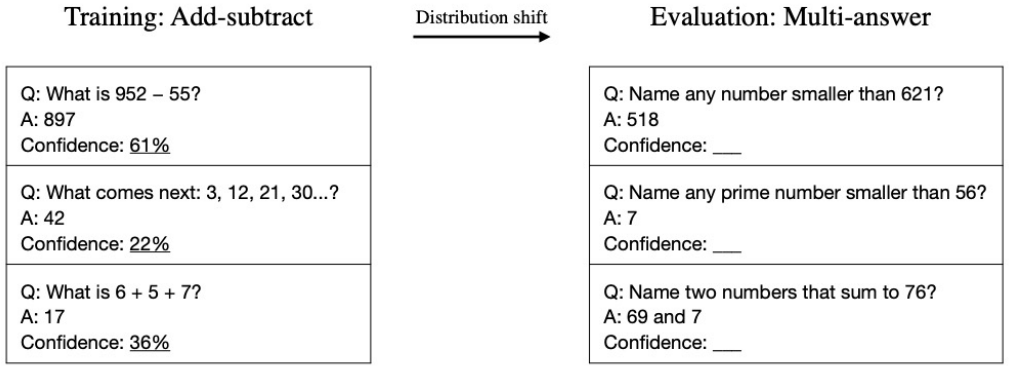
Step 2: Supervised Fine-tuning (SFT)

Input: Prompt+(q_T, a_M) Output: $\hat{p}_T$

  - **Generation (Inference)**

    Task: multi

Prompt + q_T → LLM(SFT) → A + P(Confidence)



Kind of probability	Definition	Example	Supervised objective	Desirable properties
Verbalized (number / word)	Express uncertainty in language ('61%' or 'medium confidence')	**Q: What is 952 – 55?** **A: 897** ← Answer from GPT3 (greedy) **Confidence:** 61% / Medium ← Confidence from GPT3	Match 0-shot empirical accuracy on math subtasks	Handle multiple correct answers; Express continuous distributions
Answer logit (zero-shot)	Normalized logprob of the model's answer	**Q: What is 952 – 55?** **A: 897** ← Normalized logprob for GPT3's answer	None	Requires no training
Indirect logit	Logprob of 'True' token when appended to model's answer	**Q: What is 952 – 55?** **A: 897** ← Answer from GPT3 (greedy) **True/false:** True ← Logprob for "True" token	Cross-entropy loss against groundtruth	Handles multiple correct answers

[58] Teaching Models to Express Their Uncertainty in Words, arxiv 2022

Verbalized Probability[58]

- **Task Type:** Math generation
- **Introduction:** Verbalized Probability finetunes language models to output calibrated confidence statements by supervising regression targets derived from task-level empirical accuracy estimated via repeated zero-shot answers. It quantifies prediction uncertainty, providing a calibration mechanism independent of token logits within language-model uncertainty research.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Single-pass	-	White-box	Fine-tuning	High	Calibration

[58] Teaching Models to Express Their Uncertainty in Words, arxiv 2022

P(IK)

P(IK)[46]

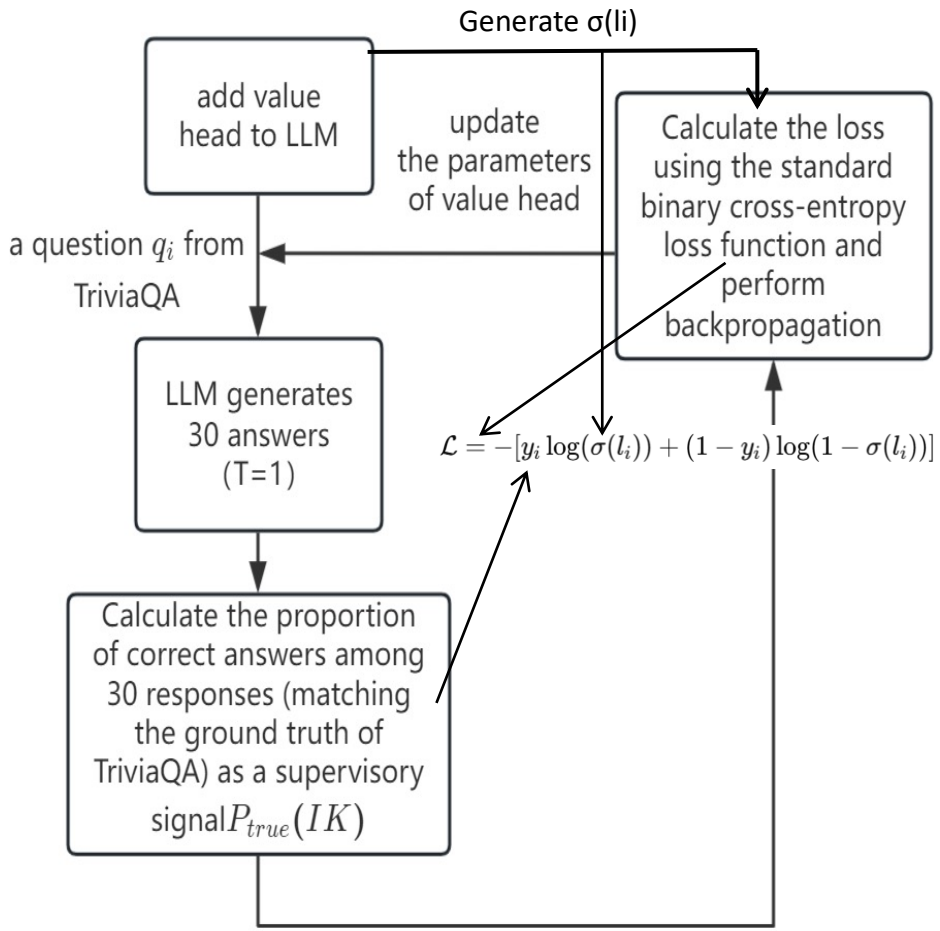
- **Task:** Estimates the probability that the model knows the correct answer before generating it.
- **Steps:**
  - Attach a scalar output layer to the transformer model.
  - This head outputs a single logit, converted via sigmoid:

$$\sigma(\text{logit}) = P(\text{IK})$$

  - Structure:

```
[Transformer Encoder/Decoder]
 ├── Language Modeling Head → predicts next token
 └── Value Head → outputs a scalar logit → σ(logit) = P(IK)
```

- **Train**



[46] Language Models (Mostly) Know What They Know, arxiv 2022

P(IK)[46]

- **Task Type:** Opening-end/Math/Code generation
- **Introduction:** A value head was added to the LLM to predict whether it "knows" the correct answer before generating it. The value head outputs a logit that is converted through a sigmoid function into the probability P(IK).

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Single-pass	-	White-box	Fine-tuning	High	Calibration/Ranking

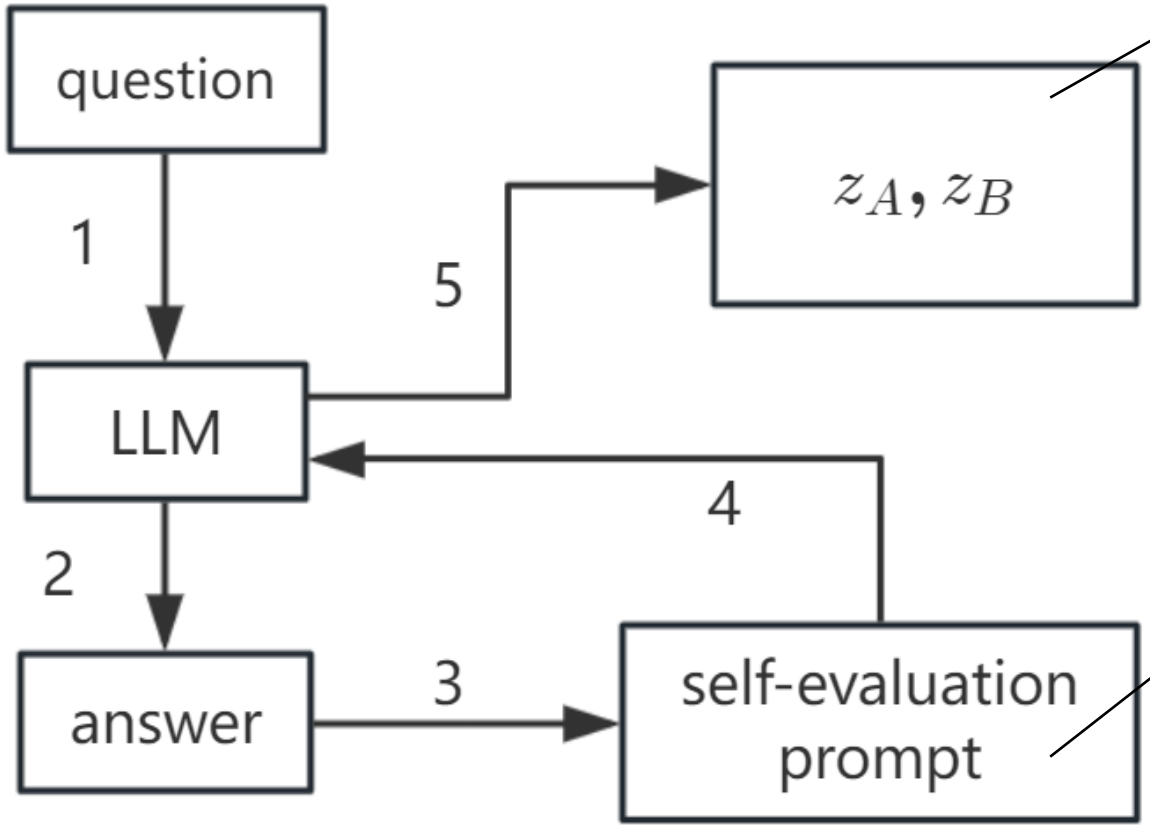
[46] Language Models (Mostly) Know What They Know, arxiv 2022

P(True)

P(True)[46]

$$P(\text{True}) = \frac{\exp(z_A)}{\exp(z_A) + \exp(z_B)}$$

where z_A = logit for token "A (True)", z_B = logit for token "B (False)".




```
Question:  Who was the first president of the United States?
Proposed Answer: George Washington was the first president.
Is the proposed answer:
 (A) True
 (B) False
The proposed answer is:
```


[46] Language Models (Mostly) Know What They Know, arxiv 2022

P(True)[46]

- **Task Type:** Opening-end/Math/Code generation
- **Introduction:** LLM assess the truthfulness of its own answer by responding to a True/False self-evaluation question. The logits assigned to each option are processed to compute the final uncertainty score.

Type	Uncertainty Sources	Model Accessibility	Training Necessity	Consumption	Evaluation
Single-pass	-	White-box	Post-hoc	Low	Calibration/Ranking

[46] Language Models (Mostly) Know What They Know, arxiv 2022